AquaSense: Real-Time Water Quality Monitoring and Prediction Using Machine Learning and IoT

21CSP302L – PROJECT

Submitted by

K. Darshan Sai [RA2311027010002]

J. Ranga Mokshagna [RA2311027010003]

P. Bharat Sai [RA2311027010007]

Under the Guidance of

Dr. V. Prasanna

Assistant Professor,

Department of Data Science and Business Systems

in partial fulfillment of the requirements for the degree

of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING
with specialization in BIG DATA ANALYTICS

DEPARTMENT OF DATA SCIENCE AND BUSINESS SYSTEMS

COLLEGE OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR- 603 203

MAY 2026

Department Of Data Science And Business Systems

SRM Institute of Science & Technology

Own Work Declaration Form

Degree / Course	:	B. Tech – Computer Science and Engineering (Big Data Analytics)
Student Name	:	K. Darshan Sai, J. Ranga Mokshagna, P. Bharat Sai
Registration Number	:	RA2311027010002, RA2311027010003, RA2311027010007
Title of Work	:	Real-Time Water Quality Monitoring and Prediction Using Machine Learning with IoT Sensor Integration

We hereby certify that this assessment compiles with the University's Rules and Regulations relating to Academic misconduct and plagiarism, as listed in the University Website, Regulations, and the Education Committee guidelines.
We confirm that all the work contained in this assessment is our own except where indicated, and that We have met the following conditions:

- Clearly referenced / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc)
- Given the sources of all pictures, data etc. that are not my own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that We have received from others (e.g.fellow students, technicians, statisticians, external sources)
- Compiled with any other plagiarism criteria specified in the Course handbook /University website

we understand that any false claim for this work will be penalized in accordance with the University policies and regulations.

DECLARATION:

we are aware of and understand the University's policy on Academic misconduct and plagiarism and we certify that this assessment is our own work, except where indicated by referring, and that we have followed the good academic practices noted above.

J. Ranga Mokshagna (RA2311027010003)	P. Bharat Sai (RA2311027010007)	K. Darshan Sai (RA2311027010002)

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

Certified that 21CSP302L - Project report titled "**AquaSense: Real-Time Water Quality Monitoring and Prediction Using Machine Learning and IOT**" is the bonafide work of **"K. Darshan Sai [RA2311027010002], J. Ranga Mokshagna [RA2311027010003], P. Bharat Sai [RA2311027010007]"** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE
DR. V. PRASANNA
SUPERVISOR
ASSISTANT PROFESSOR
DEPARTMENT OF
DATA SCIENCE AND BUSINESS SYSTEMS

SIGNATURE
DR. V. KAVITHA
PROFESSOR &HEAD
DEPARTMENT OF
DATA SCIENCE AND BUSINESS SYSTEMS

Examiner 1
Name & Signature

Examiner 1
Name & Signature

ACKNOWLEDGEMENTS

We express our humble gratitude to **Dr. C. Muthamizhchelvan**, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support.

We extend our sincere thanks to **Dr. Leenus Jesu Martin M,** Dean-CET, SRM Institute of Science and Technology, for his invaluable support.

We wish to thank **Dr. Revathi Venkataraman**, Professor and Chairperson, School of Computing, SRM Institute of Science and Technology, for her support throughout the project work.

We encompass our sincere thanks to, **Dr. M. Pushpalatha**, Professor and Associate Chairperson - CS, School of Computing and **Dr. C. Lakshmi,** Professor and Associate Chairperson -AI, School of Computing, SRM Institute of Science and Technology, for their invaluable support.

We are incredibly grateful to our Head of the Department, **Dr. Kavitha V**, Professor and Head, Department of Data Science and Business Systems, SRM Institute of Science and Technology, for her suggestions and encouragement at all the stages of the project work.

We want to convey our thanks to our Project Coordinators, Panel Head, and Panel Members Department of Computational Intelligence, SRM Institute of Science and Technology, for their inputs during the project reviews and support.

We register our immeasurable thanks to our Faculty Advisor, **Dr. V. Prasanna,** Department of Data Science and Business Systems, SRM Institute of Science and Technology, for leading and helping us to complete our course.

Our inexpressible respect and thanks to our guide, **Dr. V. Prasanna,** Department of Data Science and Business Systems, SRM Institute of Science and Technology, for providing us with an opportunity to pursue our project under her mentorship. She provided us with the freedom and support to explore the research topics of our interest. Her passion for solving problems and making a difference in the world has always been inspiring.

We sincerely thank all the staff members of Department of Data Science and Business Systems, School of Computing, SRM Institute of Science and Technology, for their help during our project. Finally, we would like to thank our parents, family members, and friends for their unconditional love, constant support and encouragement

K. Darshan Sai (RA2311027010002)

J. Ranga Mokshagna (RA2311027010003)

P. Bharat Sai (RA2311027010007)

ABSTRACT

Indeed, monitoring water quality is one of the major issues in today's world, impacting not only human life but agriculture and ecology as well. Water contaminated by different types of waste leads to multiple preventable illnesses, but currently available solutions either are too slow, costly, or restricted to lab-based environments only. This problem can be tackled by introducing AquaSense, which is a complete system for online water quality monitoring and prediction based on IoT and machine learning.

For its work, the project employs an ESP32 WROOM-32 microcontroller to constantly track three sensor readings, namely pH, NTU turbidity, and °C temperature values. This device sends information wirelessly every five seconds via WiFi protocol to the server written in Node.js, which is used for collecting information into a MongoDB collection, sending it to the Python fastapi application to run the pre-trained decision tree model, analyzing WHO thresholds and alerts, and broadcasting information to a React frontend using WebSockets.

The machine learning algorithm was developed on a dataset consisting of 31,103 samples for water quality – real samples from 21,703 observations in the Brisbane rivers as well as synthetic samples equal to 9,400 samples in total for all five quality categories. These six classifiers have been considered: Random Forest, Gradient Boosting, XGBoost, Decision Tree (pruned and unpruned), and SVM. The pruned Decision Tree classifier, having 17 leaves and depth 8, was chosen as the best model because of interpretability and performance reasons. This classifier managed to classify observations with the accuracy 99.79% with F1-score equal to 0.9979 within 0.4 seconds 43 times faster than Random Forest.

A set of features provided by the React dashboard includes animated sensor cards, Water Quality Index gauge, rolling 60 points chart with reference lines of WHO, prediction based on user-defined features, manual CSV uploading and alerts management. The application has been packaged within Docker containers for convenient one-command deployment.

TABLE OF CONTENTS

ABSTRACT		**i**
TABLE OF CONTENTS		**ii**
LIST OF FIGURES		**v**
LIST OF TABLES		**vii**
ABBREVIATIONS		**viii**
CHAPTER NO.	**TITLE**	**PAGE NO.**
1. INTRODUCTION		**1**
1.1 Introduction to Project		1
1.2 Motivation		2
1.3 Problem statement and Description		2
1.4 Sustainable Development Goal of the Project		3
2. LITERATURE SURVEY		**6**
2.1 Overview of the Research Area		4
2.2 Existing Models and Frameworks		4
2.3 Limitations Identified from Literature Survey (Research Gaps)		5
2.4 Research Objectives		7
2.5 Product Backlog (Key user stories with Desired outcomes)		8
2.5 Plan of Action (Project Road Map)		10
3. SPRINT PLANNING AND EXECUTION		**11**
3.1 Sprint 1		**11**
3.1.1 Sprint Goal		11
3.1.2 User Stories of Sprint 1		11
3.1.3 Functional Document		12
3.1.4 Architecture Document		13
3.1.5 UI Design		17

3.1.6 Functional Test Cases 19
3.1.7 Sprint Retrospective 20
3.1.8 Committed vs Completed User Stories 20
3.2 Sprint 2 21
3.2.1 Sprint Goal 21
3.2.2 User Stories of Sprint 2 21
3.2.3 Functional Document 22
3.2.4 Architecture Document 25
3.2.5 UI Design 28
3.2.6 Functional Test Cases 33
3.2.7 Sprint Retrospective 33
3.2.8 Committed vs Completed User Stories 34
3.3 Sprint 3 35
3.3.1 Sprint Goal 35
3.3.2 User Stories of Sprint 3 35
3.3.3 Functional Document 36
3.3.4 Architecture Document 39
3.3.5 UI Design 43
3.3.6 Functional Test Cases 45
3.3.7 Sprint Retrospective 45
3.3.8 Committed vs Completed User Stories 46
4. RESULTS AND DISCUSSIONS 47
4.1 Project Outcomes 47
4.2 Committed vs Completed User Stories 50
5. CONCLUSIONS & FUTURE ENHANCEMENT 54
REFERENCES 56
APPENDIX 57

A. SAMPLE CODING **57**

B. CONFERENCE PUBLICATION **61**

C. PLAGIARISM REPORT **64**

LIST OF FIGURES

CHAPTER NO	TITLE	PAGE NO.
2.1	MS Planner Board	10
2.2	Plane of Action	10
3.1	Hardware Architecture	16
3.2	System Architecture (Sprint 1)	17
3.3	Functional Test Case Document for the Sprint1	22
3.4	Sprint Retrospective for the Sprint 1	23
3.5	Bar graph for Committed Vs Completed User Stories (Sprint 1)	23
3.6	System Architecture (Sprint 2)	29
3.7	WebSocket Real-time Data Flow Sequence Diagram	29
3.8	React Dashboard Component Tree	30
3.9	Live Dashboard	33
3.10	Manual Input Page	34
3.11	History Page	35
3.12	Alerts Page	35
3.13	Functional Test Case Document for the Sprint 2	36
3.14	Sprint Retrospective for the Sprint 2	36
3.15	Bar graph for Committed Vs Completed User Stories (Sprint 2)	37
3.16	Final Complete System Architecture	42
3.17	Docker Compose Deployment Architecture	43
3.18	Six Model ML Comparison Bar Chart	45
3.19	Data Set Analysis Page	47
3.20	Functional Test Case Document for the Sprint 3	48
3.21	Sprint Retrospective for the Sprint 3	48
3.22	Bar graph for Committed Vs Completed User Stories (Sprint 3)	49

4.1	Confusion Matrix for Decision Tree (pruned)	51
4.2	Feature Importance Chart	52

LIST OF TABLES

CHAPTER NO	TITLE	PAGE NO.
2.1	User Stories	8
3.1	Detailed User Stories of sprint 1	11
3.2	Access level Authorization Matrix (Sprint1)	15
3.3	Technology Stack	19
3.4	API Endpoints	20
3.5	Detailed User Stories of sprint 2	24
3.6	Access level Authorization Matrix (Sprint2)	27
3.7	Detailed User Stories of sprint 2	38
3.8	Access level Authorization Matrix (Sprint3)	41
3.9	Final Port Summary	45
4.1	Comparative Performance of All ML Classifiers	50
4.2	Per-Class Metrics for Decision Tree (pruned)	51
4.3	System Performance Metrics	52
4.4	Overall Committed vs Completed User Stories	53

ABBREVIATIONS

IoT	Internet of Things
ML	Machine Learning
DT	Decision Tree
WHO	World Health Organization
WQI	Water Quality Index
NTU	Nephelometric Turbidity Unit
ESP32	Espressif Systems 32-bit Microcontroller
API	Application Programming Interface
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
GPIO	General Purpose Input Output
ADC	Analog to Digital Converter
SVM	Support Vector Machine
RF	Random Forest
XGB	XGBoost Classifier
GBM	Gradient Boosting Machine
CV	Cross Validation
F1	F1 Score (Harmonic Mean of Precision and Recall)
WQI	Water Quality Index
BIS	Bureau of Indian Standards
MQTT	Message Queuing Telemetry Transport
REST	Representational State Transfer

CHAPTER 1
INTRODUCTION

1.1 Introduction to the Project

Water is essential for life. However, millions of people across the globe consume water that poses risks to their health due to the presence of bacteria, industrial waste, agricultural contaminants, or unbalanced minerals. According to the World Health Organization, at least two billion people utilize water from sources contaminated with fecal matter, making waterborne diseases one of the primary reasons for fatalities in underdeveloped areas.

Conventional methods of water testing involve sample collection followed by laboratory analysis. Although precise, such techniques are too time-consuming and costly to provide real-time information about water quality. Therefore, what society needs is a technology capable of alerting users immediately if their water supply is compromised.

AquaSense came about to solve this problem. AquaSense is a fully-integrated, real-time water quality monitoring and prediction system combining hardware, machine learning, and the latest web dashboard technology. At the heart of this system sits an ESP32 microcontroller, connected to three actual sensors (pH, turbidity, and temperature). The microcontroller monitors the state of the water by taking readings every five seconds and sending this information over wirelessly. A Node.js backend takes this raw data, which gets processed by a trained Decision Tree ML model that predicts one out of the five WHO water quality classes in real time. Finally, the data is displayed on a React web dashboard.

But what makes AquaSense truly remarkable is not only its high precision (99.79%), but rather explainability. With Decision Trees, the logic behind how water is classified can be easily understood, with a series of simple if statements. For example, water with a turbidity lower than 4 NTU and pH levels ranging from 6.8 to 7.4 is classified as Excellent quality water, following the WHO guidelines.

1.2 Motivation

Inspiration for undertaking this project was the following thought: there is already the technology to monitor water quality available, however, it has not yet been made available to those who actually require it the most. State-of-the-art measuring devices are present in state labs while people living in various places have no possibility to ensure that the water that comes out of their borewell or piped connection is safe.

The emergence of inexpensive IoT controllers such as ESP32 and affordable pH and turbidity sensors now makes it possible to assemble a decent water quality monitoring node at a cost less than ₹2,000. The challenge was therefore the lack of an effective software layer that would be able to gather data from multiple devices, analyze it with machine learning algorithms and show the information to users.

Moreover, this project was also inspired by the chance to get deeper insight into the connection between hardware and software technologies including cloud-based databases and machine learning algorithms. Such a project is definitely interdisciplinary as the areas of embedded systems, cloud computing and machine learning intersect significantly.

1.3 Problem Statement and Description

There are three basic flaws inherent in existing methods of water quality testing:

Firstly, they are reactive, not proactive. The laboratory analysis is done only if there is suspicion of a contamination event, not as a precautionary action beforehand. People could already have fallen ill by the time the problem was identified.

Secondly, they are sporadic in nature. One test done per week or month would not account for the fluctuations in water quality due to rainfall, industrial runoff, or seasonal trends.

Thirdly, they lack a meaningful classification system. IoT devices collect one metric and show its numerical value without translating it into a user-friendly evaluation of whether or not the water is safe to drink.

In contrast, AquaSense tackles all three issues. It tests continuously, every five seconds, and employs a machine learning algorithm to classify the water quality into five user-friendly classes: Excellent, Good, Poor, Very Poor, and Unsafe. It raises an automatic alarm if any threshold levels are breached, and the results are displayed live in an easy-to-read dashboard interface.

1.4 Sustainable Development Goal of the Project

The AquaSense project helps address UN Sustainable Development Goal 6: Clean Water and Sanitation. The UN SDG 6 seeks to ensure availability and sustainable management of water and sanitation for all by 2030. In addition to policy measures and infrastructure development, this goal requires that accurate water quality measurement and monitoring solutions be available.

The affordability and practicality of AquaSense enable it to serve as one of the key elements of the necessary infrastructure for meeting UN SDG 6. The deployment of the technology can be done across water treatment facilities, boreholes, river streams, dams, and irrigation channels.

Additionally, UN SDG 3: Good Health and Well-Being can be achieved due to the disease prevention enabled through the water quality monitoring offered by AquaSense. UN SDG 9: Industry, Innovation, and Infrastructure is also addressed due to the use of IoT and AI technologies within the project.

CHAPTER 2

2.1 Overview of the Research Area

Water quality monitoring has long been important in environmental science and public health. Traditionally, water is tested in laboratories for parameters like pH, turbidity, and contaminants. Although accurate, this method is slow, costly, and cannot provide continuous monitoring, making it unsuitable for real-time detection of water quality changes.

With the introduction of low-cost sensors and microcontrollers, especially during the IoT era, real-time water monitoring became possible. Devices like ESP8266 and ESP32 enabled affordable systems that can continuously collect and transmit water quality data, making such solutions accessible even in rural areas.

In addition, machine learning has improved water quality analysis by considering multiple parameters together rather than using simple threshold values. This allows more accurate classification of water conditions.

Overall, IoT-based water monitoring combined with machine learning is an interdisciplinary field that integrates embedded systems, environmental science, and data science. It enables efficient, real-time, and cost-effective solutions for ensuring safe water quality.

2.2 Existing Models and Frameworks

IoT-based water quality monitoring and machine learning studies can be categorized into four categories: IoT system, ML model, hybrid IoT+ML system, and deployment framework.

Previous IoT systems consisted of Arduino with sensors such as pH and turbidity, where communication was conducted using GSM technology. In subsequent systems, the use of ESP8266/ESP32 with WiFi and MQTT resulted in lower costs and power usage. Nevertheless, these previous systems only involved binary classification (safe/unsafe), and analysis was basic. Advanced IoT systems using Raspberry Pi allowed for multi-sensor integration but had higher costs and power requirements.

Regarding ML models, SVM, Decision Tree, Random Forest, and ANN algorithms have been utilized extensively to classify water quality. Previous studies indicate that SVM and ensemble models produced highly accurate results, but they required extensive training data and a large number of input variables, making them unsuitable for affordable IoT systems. On the other hand, Decision Tree models produced reliable results without requiring many features.

IoT and ML are combined in some of the latest papers, which allows for real-time classification. This kind of application uses edge computing technology, but it lacks capabilities such as web dashboards, data storage, and remote access.

Docker and microservices are used in today's applications to facilitate integration and management of database systems, ML models, and dashboards among others.

2.3 Limitations Identified through Literature Survey (Research Gaps)

Upon reviewing existing systems from the literature, it becomes clear that there are a number of limitations consistently found in all of the published work, and which have not been effectively dealt with in any published system. These research gaps are what has driven the development of AquaSense.

Gap 1 - Lacks Multi-Class WHO-Standard Classification of Quality

Almost all IoT water quality detection systems found in the literature use binary classifications for water quality, wherein the water is either safe/unsafe or potable/non-potable. This type of binary classification does not provide adequate details needed for making decisions in relation to water. If a water authority must determine if water is fit for human consumption, it must be known not only if the water is potable or unsafe, but how potable it is and which way the trend is going.

Gap 2 - Unexplainable ML Models

The majority of published solutions rely heavily on black-box models - Random Forest ensemble, artificial neural networks, or SVM with RBF kernel - where inner rules of decision-making processes remain unexplainable and invisible for inspection. It creates severe limitations for applications focused on ensuring water safety. Water agency cannot declare water safe using a solution with no way to verify its rules. The AquaSense system closes this gap with the use of a pruned Decision Tree

as the chosen machine learning algorithm that produces seven explainable classification rules aligning with WHO recommendations for drinking water.

Gap 3 - Absence of Web Dashboard with WebSocket Communication

None of the available solutions provide an interface for displaying real-time water quality information using a real-time live web dashboard with WebSocket technology implemented in such a way that it provides real-time updates of all displayed visual elements at once. The AquaSense system includes a React-based web interface communicating with backend using WebSocket technology provided via Socket.io library.

Gap 4 - Lack of IoT and Manual Input Separation in Live Monitoring

Current systems that enable automatic inputs and manual inputs do not distinguish these two different sources in the live monitoring section. The use of manual input predictions and actual sensor inputs at the same time may cause misunderstandings about the status of water quality. AquaSense strictly separates the two sources of information in three layers so that no manual prediction affects live IoT sensing.

Gap 5 - Lack of Dataset Predicting Function

None of the reviewed systems offer batch predicting function by uploading historical data and visualizing the results. This kind of ability may be helpful for authorities to analyze the previous sensors' data or for researches to check their validity in contrast with known dataset. Four chart options of the AquaSense's "Dataset Predict" function can solve this problem.

Gap 6 - No or Insufficiently Automated Deployment Using Containers

Most existing IoT ML algorithms use time-consuming deployment procedures that involve setting up different Python environments, installing Node.js separately, databases, and dependencies related to ML. AquaSense solves this problem using Docker Compose for containerized deployment that cuts down on deployment time from 45 minutes of manual work to less than 3 minutes.

Gap 7 - Limited Training Set Size

There exist IoT classification algorithms whose ML models have been trained using only artificial or small laboratory training data (with fewer than 5,000 samples). This poses an increased risk of overfitting, resulting in less robust generalization to real-world conditions where sensors can be used. The algorithm presented in this paper uses a combination of real laboratory and artificially generated training data, including 21,703 real measurement points collected by sensors installed at Brisbane river monitoring stations and 9,400 artificial measurement points, totaling 31,103.

2.4 Research Objectives

From the above literature review, several constraints were outlined, which AquaSense aimed to fulfill in order to solve seven research problems. These research problems were as follows:

Objective 1 - Implementation of an affordable IoT sensor node based on the ESP32 WROOM-32 microcontroller equipped with a pH, turbidity, and temperature sensor, which can continuously measure water quality and wirelessly transmit its information every five seconds via standard WiFi networks.

Objective 2 - Creation of a multi-class water quality classification model that classifies water quality into five categories (Excellent, Good, Poor, Very Poor, and Unsafe) according to the WHO classification criteria based on the three water quality attributes measured by inexpensive IoT sensors, having more than 95% classification accuracy.

Objective 3 - Comparison and evaluation of performance of six machine learning classifiers (Random Forest, Gradient Boosting, XGBoost, Decision Tree with all nodes, pruned Decision Tree, and SVM with RBF kernel) using stratified five-fold cross-validation.

Objective 4 - Choose and support an interpretable ML model that is able to be validated by comparison with the World Health Organization drinking water guidelines, without any knowledge of machine learning required, thereby maintaining the transparency and auditability of the system.

Objective 5 - Develop an instant web interface utilizing WebSocket technology for live push notifications, WHO alert thresholds, animation visualization of WQI, and strict segregation between

IoT sensor data and manually entered data, all being updated within 100 milliseconds of each new sensor entry.

Objective 6 - Provide batch prediction capabilities by allowing for uploading of CSV files with historical sensor data, including ML classification results, statistical information, and four interactive graphs for visualization.

Objective 7 - Containerize the entire application for deployability using Docker Compose. This will enable the entire four-service architecture (MongoDB, ML service, Node.js backend, React frontend) to be deployed on any computer running Docker Desktop in less than five minutes with one command line entry.

2.5 Product Backlog

Table 2.1 User Stories

S. No	User Story ID	User Story Description
1	US #1	As a system, I want to automatically receive pH, turbidity, and temperature data from the ESP32 sensor every 5 seconds so that water quality can be monitored continuously without any manual effort.
2	US #2	As a water authority operator, I want the system to automatically classify each reading into WHO categories (Excellent, Good, Poor, Very Poor, Unsafe) so that I can quickly understand the water safety level.
3	US #3	As a dashboard user, I want to view live sensor values, WQI score, and quality classification in real time without refreshing the page so that I always see the latest updates.
4	US #4	As an operator, I want to receive automatic alerts when any parameter crosses WHO safe limits so that I can take immediate action to prevent risks.

5	US #5	As a field engineer, I want to manually enter sensor values in the dashboard and get instant ML predictions so that I can test the system even without physical sensors.
6	US #6	As a researcher, I want to upload a CSV file with historical sensor data and receive batch predictions along with charts and downloadable results so that I can analyze large datasets efficiently.
7	US #7	As a manager, I want to view historical sensor readings and predictions in a paginated table so that I can easily analyze past water quality trends.
8	US #8	As an administrator, I want to deploy the entire system using a single Docker command so that setup becomes quick and easy across different environments.
9	US #9	As a decision-maker, I want the ML model to provide confidence scores along with predictions so that I can understand how reliable each result is.
10	US #10	As a developer, I want the system to switch to a WHO rule-based method if the ML model fails so that the system continues to function without interruption.
11	US #11	As an IoT device, I want the ESP32 to automatically reconnect to WiFi and resume data transmission after disconnection so that monitoring remains continuous.

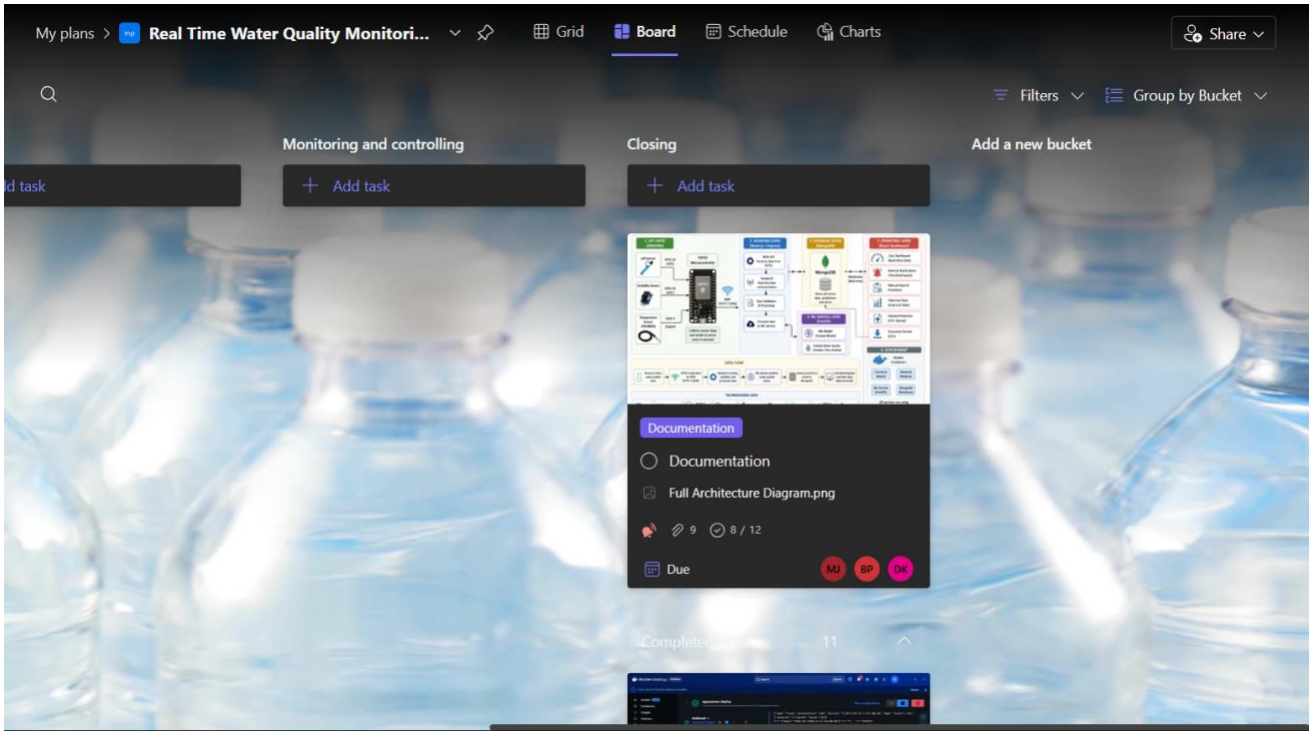


Fig 2.1 Ms Planner

2.1 Plan of Action

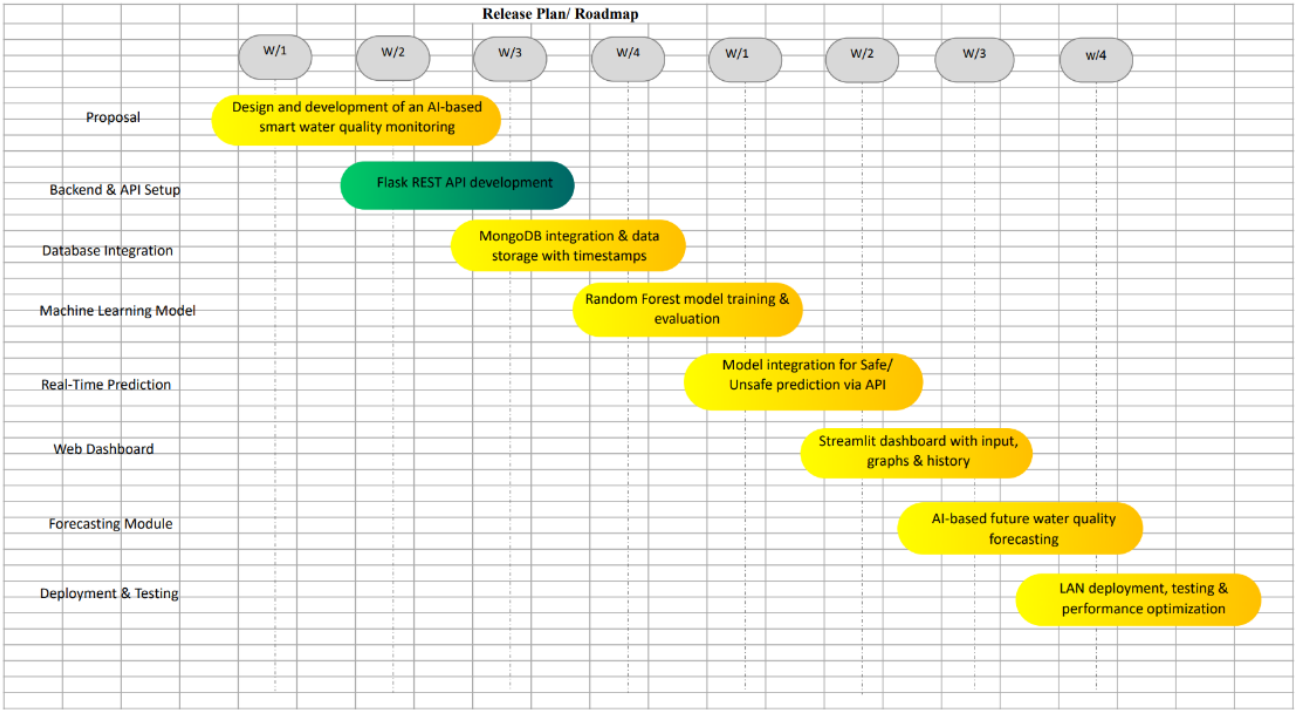


Fig 2.2 Plan of Action

CHAPTER 3

SPRINT PLANNING AND EXECUTION

3.1 Sprint 1

3.1.1 Sprint Goal

The goal of Sprint 1 was to build the core system pipeline for AquaSense.

- Develop ESP32 firmware to collect sensor data (pH, turbidity, temperature)
- Implement backend API using Node.js
- Integrate MongoDB for data storage
- Build ML pipeline and train initial model
- Enable end-to-end data flow from sensor → backend → ML → database

3.1.2 User Stories of Sprint 1

Table 3.1 Detailed User Stories of sprint 1

S. No	User Story ID	User Story Description
1	US 1	As a system, I want to collect sensor data from the ESP32 using the HTTP POST method every 5 seconds so that water quality can be monitored continuously.
2	US 2	As a machine learning model, I want to classify each observation into WHO quality classes and provide the output within 50 ms.
3	US 3	As an IoT device, I want to automatically reconnect if a WiFi disconnection occurs so that data transmission continues without interruption.
4	US 10	As a developer, I want the system to switch to a WHO rule-based method if the ML model fails so that the system continues to function without interruption.

5	US 11	As an IoT device, I want the ESP32 to automatically reconnect to WiFi and resume data transmission after disconnection so that monitoring remains continuous.

3.1.2Functional Document

3.1.2.1Introduction

The AquaSense - IoT based Water Quality Monitoring System is an intelligent and real time water quality monitoring system based on Internet of Things sensors and machine learning. The system comprises of ESP32 based sensors, a backend API, a machine learning prediction service and a real-time dashboard.

What this project is about:
Gathering real-time environmental data (pH, turbidity, temperature)
Predicting water quality by processing data through ML models
Live monitoring and alerts delivered in a dashboard
Allow for batch prediction and data analysis

Functional testing makes sure that the whole ecosystem (IoT, backend, ML, and frontend) is functioning properly and smoothly.

3.1.2.2Product Goal

AquaSense aims to develop an intelligent water quality monitoring system to provide accurate and immediate information.

The system is designed to:

- IoT sensors to monitor water quality
- Machine learning models for water quality prediction
- WebSocket for real-time dashboard updates
- Generate alerts according to WHO standards
- Enable CSV upload for batch prediction
- Deploy with Docker with confidence

3.1.2.3Demography (Users, Location)

Users

Environmental monitoring bodies

Researchers and students

Water quality analysts, industrial

Intelligent city managers

User Characteristics

from basic to advanced technical skills

The need for real-time monitoring and analysis.

Data driven decision making

Location

Target: Global

Focus: Regions with IoT adoption and water monitoring requirements

3.1.2.4Business Processes

i. **Data Acquisition (IoT Layer)**

- The ESP32 acquires sensor data (pH, turbidity, temperature)
- Transmits sensor data to the backend through HTTP POST request

ii. **Data Analysis (Backend and Machine Learning)**

- The backend stores sensor data in the MongoDB database
- Machine learning model predicts:
- Water quality class
- WQI score
- Confidence score

iii. **Real-time Monitoring**

WebSocket (Socket.io) provides real-time updates

Displays:

- Sensor data

- WQI score
- Alerts

iv. Batch Processing

- Users upload CSV files
- Batch processing of many records
- Displaying results and exportable

v. Alert System

- Alerts generated when thresholds are exceeded
- Alert display and resolution

3.1.2.5 Features

Feature 1: IoT Data Collection

Description: IoT data is collected by ESP32 module and sent to the backend API.

User Story: As a system, I need real-time collection of water data to monitor water quality constantly.

Feature 2: ML Prediction

Description: ML model predicts water quality category and WQI value.

User Story: As a user, I need precise predictions to know water safety level.

Feature 3: Real-Time Dashboard

Description: Real-time sensor data visualization is done via WebSocket.

User Story: As a user, I need real-time updates to be aware of changes immediately.

Feature 4: Alerting Feature

Description: Alerts are issued whenever the values exceed the thresholds by WHO.

User Story: As a user, I need alerts to take necessary actions quickly.

Feature 5: Batch Prediction (Upload CSV File)

Description: Users upload CSV files to make bulk predictions on data.

User Story: As a user, I need to upload multiple records at once for quick analysis.

Feature 6: Data Filtering/Separation

Description: Data received from IoT devices is separated from manually entered data.

User Story: As a user, I need my data displayed correctly to avoid any confusions.

Feature 7: Docker Deployment

Description: The system is deployed using Docker Compose with several services.

User Story: As a developer, I would like simple deployment so that the system works well on different machines.

3.1.2.6 Authorization Matrix

Table 3.2 Access level Authorization Matrix

Role	Access Level
Administrator	Full access to system, database, and services
Analyst/User	Access to dashboard, predictions, alerts
IoT Device	Can send sensor data via API
Guest	Limited access (view-only dashboard if enabled)

3.1.2.7Assumptions

- Sensors are calibrated and give accurate data
- Model is trained using an adequate set of data
- Services are available all the time (elsewhere available)
- Internet connectivity is available for all users
- Docker environment is properly configured
- Classifications follow WHO standards

3.1.3 Architecture Document

3.1.3.1 Introduction

The architecture of the AquaSense system created in Sprint 1 creates an entire pipeline for the collection, analysis, and processing of the data related to water quality. The system uses IoT hardware, backend services, and machine learning models to make sure that all collected information is correct and relevant.

After Sprint 1, our system was capable of doing the following:

- Collecting sensor data via hardware
- Processing data via API services
- Making predictions based on machine learning models
- Storing results into the database

3.1.3.2 Hardware Architecture

Hardware architecture involves an ESP32 microcontroller that communicates with three different sensors:

- pH sensor -> GPIO35 (analog input)
- Turbidity sensor -> GPIO34 (analog input)
- DS18B20 temperature sensor -> GPIO4 (digital input, one-wire protocol)
- Pull up resistor value for DS18B20 is 4.7k ohms.

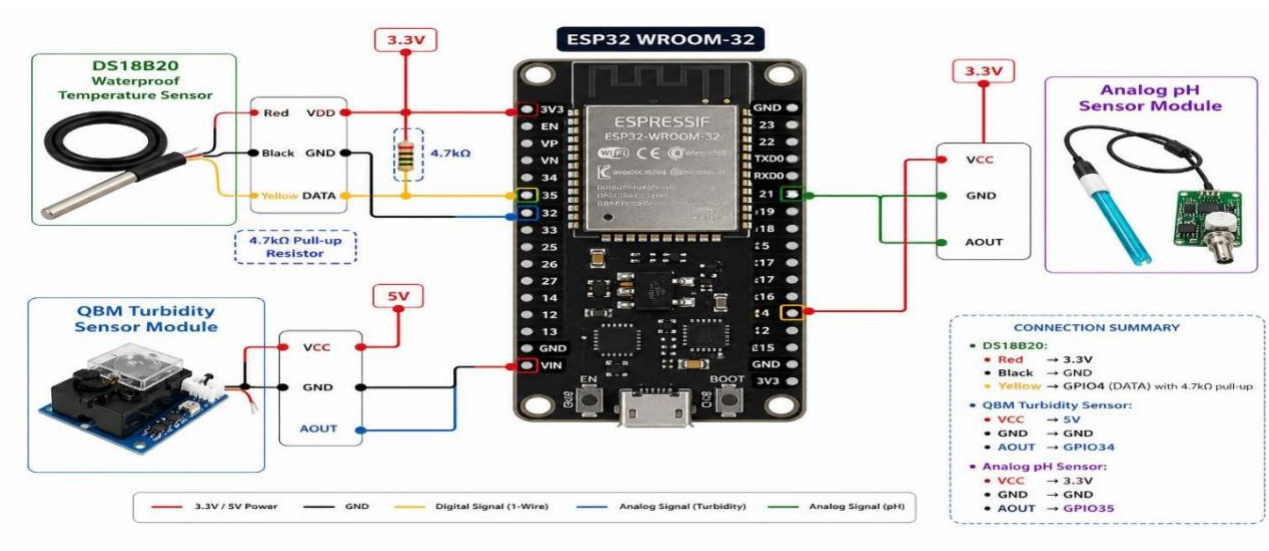


Fig 3.1 Hardware Architecture

Features

- Reads sensor data every 5 seconds
- Uses noise reduction technique by averaging
- Sends data to backend through HTTP request

3.1.3.3 System Architecture

The architecture has three layers:

1) IoT Layer (Data Source)

   ESP32 senses and transmits sensor values

2) Backend Layer (Data Processor)

   Node.js API receives data from ESP32

   Invokes ML service

   Validates data using rules

   Stores results in database

3) Machine Learning Service Layer

   Python FastAPI microservice

   Feature engineering

   Predicts water quality with Decision Tree algorithm

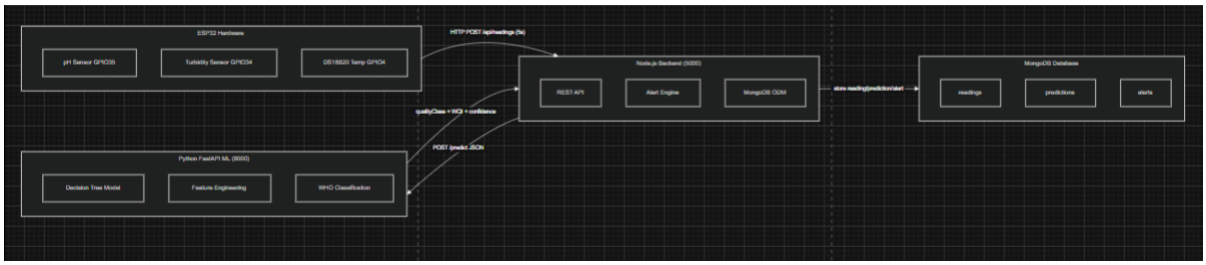


Fig 3.2 System Architecture (Sprint 1)

3.1.3.4 Data Flow Explanation

The process works as follows:

1. The ESP32 takes readings for pH, turbidity, and temperature sensors
2. Averages multiple samples to minimize error
3. Converts to JSON format
4. Sends POST request to backend (API endpoint /readings)
5. The backend verifies API key and data structure
6. The backend invokes ML service (/predict)
7. The ML service does:
8. Feature engineering
9. Decision Tree prediction
10. Output includes:
11. Water quality class
12. Water Quality Index score
13. Confidence value
14. The backend checks WHO guidelines
15. Raises alarms if required
16. Stores data in MongoDB
17. Backend responds with HTTP 201 status to ESP32

3.1.3.5 Machine Learning Pipeline

Components of ML pipeline include:

Data Cleaning -> Removing outliers and missing values

Feature Engineering -> From three to eight features created

Train-Test Split -> 80% train, 20% test split

Model Building -> Random Forest, Gradient Boosting, XGBoost, Decision Tree, SVM

Model Selection -> Decision Tree selected

Model Saving -> Saved using JobLib

Final Model

Algorithm -> Decision Tree

Accuracy -> 99.79%

Training time -> 0.4 second

3.1.3.6 Technology Stack

Table 3.3 Technology Stack

Layer	Technology
Hardware	ESP32 WROOM-32, DS18B20, Turbidity Sensor, pH Sensor
Firmware	C++ (Arduino IDE), WiFi.h, HTTPClient.h, ArduinoJson
Backend	Node.js, Express.js, MongoDB, Mongoose
ML Service	Python, FastAPI, scikit-learn, joblib
Communication	HTTP (REST API)

3.1.3.7 Database Schema

i. Readings Collection

- deviceId (String)
- ph (Float)
- turbidity (Float)
- temperature (Float)
- source (String)
- timestamp (Date)

ii. Predictions Collection

- readingId (Reference)
- qualityClass (String)
- wqiScore (Float)
- confidence (Float)

- modelVersion (String)
- timestamp (Date)

iii. Alerts Collection

- readingId (Reference)
- parameter (String)
- value (Float)
- threshold (Float)
- severity

3.1.4.8 API Endpoints

Table 3.4 API Endpoints

Method	Endpoint	Description
POST	/api/readings	Send sensor data and trigger prediction
GET	/api/readings	Retrieve paginated readings
GET	/api/predictions	Retrieve prediction history
GET	/api/alerts	Retrieve alerts
GET	/health	Check system health

3.1.4.9 Security

- x-api-key based security implementation
- HTTP error code returned when unauthorized access occurs - HTTP 401
- Rate limiting implemented (max of 100 requests per minute per IP).

3.1.4 User Interface Design (Sprint 1)

3.1.4.1 Introduction

No user interface was created in Sprint 1, with emphasis placed on developing the main software architecture with hardware support, API and ML model.

The only interface that existed was the Arduino Serial Monitor displaying sensor inputs and prediction outputs in real-time mode.

3.1.4.2 Serial Monitor Display

Arduino Serial Monitor served as the temporary display interface set to the 115200 baud rate.

- Display Data
- pH value
- Turbidity (NTU)
- Temperature (°C)
- HTTP response status
- Water quality prediction (Quality + WQI)
- Alerts counter

Output Sample Format

==========

AquaSense ESP32 v1.0

==========

[#1] pH: 7.24

[#1] Turbidity: 1.52 NTU

[#1] Temperature: 24.3 C

[OK] Quality=Good WQI=85 Alerts=0

This ensured proper readability of the data display on the serial monitor interface after each cycle.

3.1.4.3 Dashboard Layout Design (Wireframe)

A wireframe design for the future dashboard with tabbed sections:

Live Dashboard -> Sensor data display, WQI gauge, alerts display

Manual Input -> Form input + predicted result

Dataset Prediction -> File upload + prediction results

History -> Paginated table data display

Alerts -> Warning messages display with solve button

3.1.4.4 Design Considerations

Theme: Dark UI color theme using teal (default), amber (warning), red (emergency)

Typography: Readable fonts with large-sized sensor values

Layout: Grid space (16px) using card layouts

3.1.5 Functional Test Cases

Test Case ID	Test Description	Expected Result	Status	Remarks
TC01	ESP32 sends valid JSON payload to /api/readings	Backend returns HTTP 201 with prediction result	PASS	Tested with simulator
TC02	ML service predicts quality class for pH = 7.2, turbidity = 1.5, temp = 24	Returns **"Good"** with WQI score ~85	PASS	—
TC03	MongoDB stores reading with all required fields	Document created with timestamp, pH, turbidity, temperature, source	PASS	—
TC04	Backend falls back to rule-based scorer when ML service is down	Returns quality classification without error	PASS	ML service killed manually
TC05	ESP32 reconnects to WiFi after disconnection	Resumes sending readings within 30 seconds	PASS	WiFi toggled off and on
TC06	pH sensor reads correct value in pH 7.0 buffer solution	Reading shows 7.0 ± 0.1 after calibration	PASS	Calibrated with buffer solution
TC07	Turbidity sensor reads 0 NTU in clear distilled water	Reading shows < 0.5 NTU	PASS	—
TC08	DS18B20 reads correct temperature at room temperature	Reading shows 25°C ± 0.5°C	PASS	Verified with thermometer
TC09	API key authentication rejects unauthorized request	Returns HTTP 401 Unauthorized	PASS	Tested without x-api-key header
TC10	GET /api/readings returns paginated list of readings	Returns 20 readings per page with total count	PASS	—

Figure 3.3 Functional Test Case Document for the Sprint 1

3.1.6 Sprint 1 Retrospective

Liked	Learned	Lacked	Longed For
ESP32 firmware was completed ahead of schedule, helping faster integration with backend and ML pipeline.	Learned how to integrate ESP32 with backend APIs and connect it with the ML prediction pipeline.	Initial turbidity sensor calibration took longer due to voltage-to-NTU mapping challenges.	A more structured sensor calibration approach to reduce setup time.
ML model achieved 99.79% accuracy on first run, boosting team confidence in dataset and feature engineering.	Gained experience in handling real-time IoT data and processing it efficiently.	Missing 4.7kΩ pull-up resistor for DS18B20 caused hardware delay.	Better hardware planning and complete component checklist before starting.
MongoDB schema design was clean and required no changes in later sprints, ensuring smooth development.	Learned proper database schema design for IoT-based applications.	WiFi connectivity issue due to incorrect IP configuration in ESP32 firmware.	More robust network handling and debugging strategy for IoT devices.
Overall system integration (IoT + Backend + ML) worked smoothly with effective team collaboration.	Learned debugging techniques for connectivity issues and firmware-level fixes.	Lack of proper documentation for sensor calibration process.	Clear documentation and standard procedures for calibration and setup.
System stability was maintained after resolving connectivity issues.	Learned importance of fallback mechanisms (rule-based system when ML fails).	No separation between manual input and IoT sensor data in pipeline.	Better data pipeline design with clear separation of data sources and real-time updates (Socket.io).

Figure 3.4 Sprint Retrospective for the Sprint 1

2.1.7Committed vs Completed User Stories

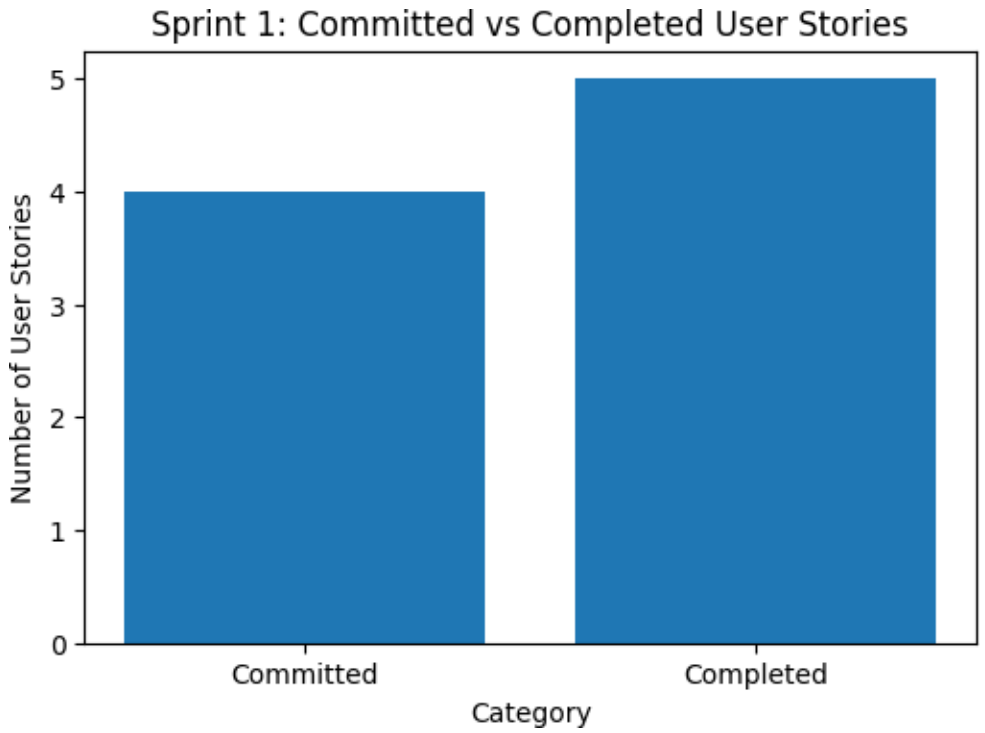


Figure 3.5 Bar graph for Committed Vs Completed User Stories (Sprint 1)

3.2 Sprint 2

3.2.1 Sprint Goal

The goal of Sprint 2 was to create a real-time user interface and interactive features.

- Develop React dashboard with 5 tabs
- Implement WebSocket (Socket.io) for live updates
- Add WHO-based alert system
- Enable manual input prediction
- Display historical data with pagination

3.2.2User Stories of Sprint 2

Table 3.5 Detailed User Stories of sprint 2

S. No	User Story ID	User Story Description
1	US 3	As an IoT device, I want to automatically reconnect if a WiFi disconnection occurs so that data transmission continues without interruption.
2	US 4	As a dashboard user, I want to view sensor values and quality classes in real time without refreshing the browser so that I always see the latest data.
3	US #5	As an operator, I want WHO thresholds with severity levels to be generated automatically so that I can take action when water quality deteriorates.
4	US #7	As a manager, I want to view historical sensor readings and predictions in a paginated table so that I can easily analyze past water quality trends.

3.2.3 Functional Document

3.2.3.1 Introduction

In Sprint 2, the development work included the creation of a fully-fledged interface for the AquaSense Water Quality Monitoring System along with real-time communication integration.

Sprint 2 included:

- React-based dashboard
- WebSocket (Socket.io) real-time updates
- WHO standards-based alert generation system
- Manual input prediction module
- Visualization of historical data

Testing was conducted to verify functionality of all interface components, real-time updates and system interaction.

3.2.3.2 Product Objective

Sprint 2 was geared towards improving system usability through inclusion of real-time monitoring interface and interactivity elements.

System objectives include:

- Displaying real-time data without refreshing
- Alert generation according to WHO guidelines
- Enabling users to enter their own manual inputs for prediction
- Historical data visualization with pagination
- Ensuring smooth real-time communication

3.2.3.3 Demography (Users, Location)

Users

- Environmental analysts
- Research scholars/students
- System administrators

User Attributes

- Real-time monitoring needs

- Easy visualization requirements
- Medium level of technical skills

Location

- Used across the globe
- Use within smart monitoring environments

3.2.3.4 Business Processes

1. Real-Time Data Monitoring

- Data sent to backend by sensors
- Updates are pushed to dashboard using WebSocket
- Instant updates without page refreshes

2. Alerts Creation

- Back-end analyzes WHO standards
- Alerts created and shown in UI
- Can be handled by users

3. Manual Prediction

- Users provide data such as pH, turbidity, temperature
- Machine learning model produces predictions
- Instant results are displayed

4. Handling Historical Data

- All data saved in database
- Presented in pages table
- Contains IoT and manually inputted data

3.2.3.5 Features

Feature 1: Real-Time Dashboard

Description: Shows real-time data through WebSockets.

User Story: As a user, I need real-time updates so that I can monitor the quality of the water in real time.

Feature 2: Alert System

Description: Provides alerts whenever values pass WHO's threshold.

User Story: As a user, I want an alert system so that I can act in case the water is not fit for consumption.

Feature 3: Prediction from Manual Input

Description: Provides predictions based on manually entered values.

User Story: As a user, I want to enter different values and predict the quality of the water.

Feature 4: Historical Readings Table

Description: Provides historical reading in tabular form along with pagination.

User Story: As a user, I want to analyze my historical readings.

Feature 5: WebSocket Connection

Description: Maintains live connection between front and back end.

User Story: As a user, I want to have live updates.

3.2.3.6 Authorization Matrix

Table 3.6 Authorization Matrix

Role	Access Level
Administrator	Full access to dashboard and alerts
User	Access to dashboard, predictions, and history
IoT Device	Sends sensor data via API
Guest	Limited (view-only if enabled)

3.2.3.7 Assumptions

- WebSocket connection remains stable
- Backend and ML services are running
- Sensor data is accurate and timely

- WHO thresholds are correctly defined
- Browser supports modern JavaScript features

3.2.4 System Architecture Document (Sprint 2)

3.2.4.1 Introduction

In sprint 2, the AquaSense system architecture is expanded to include the React dashboard and webSocket (socket.io).

As opposed to having 3-layers of architecture pipeline in sprint 1, we now have 4-layers of architecture with real-time data transmission between the backend and frontend.

3.2.4.1 System Architecture

The system includes four distinct layers.

1. **IoT Layer (ESP32)**
   Collects sensor readings (pH, Turbidity, temperature)
   HTTP POST request sent
2. **Backend Layer (Node.js)**
   REST API + Alert Engine
   Data stored in MongoDB
   Websocket event emitter
3. **ML Service (FastAPI)**
   Predicts the quality of the water
   WQI score output
4. **Frontend Layer (React Dashboard)**
   Real-time data displayed
   User interface

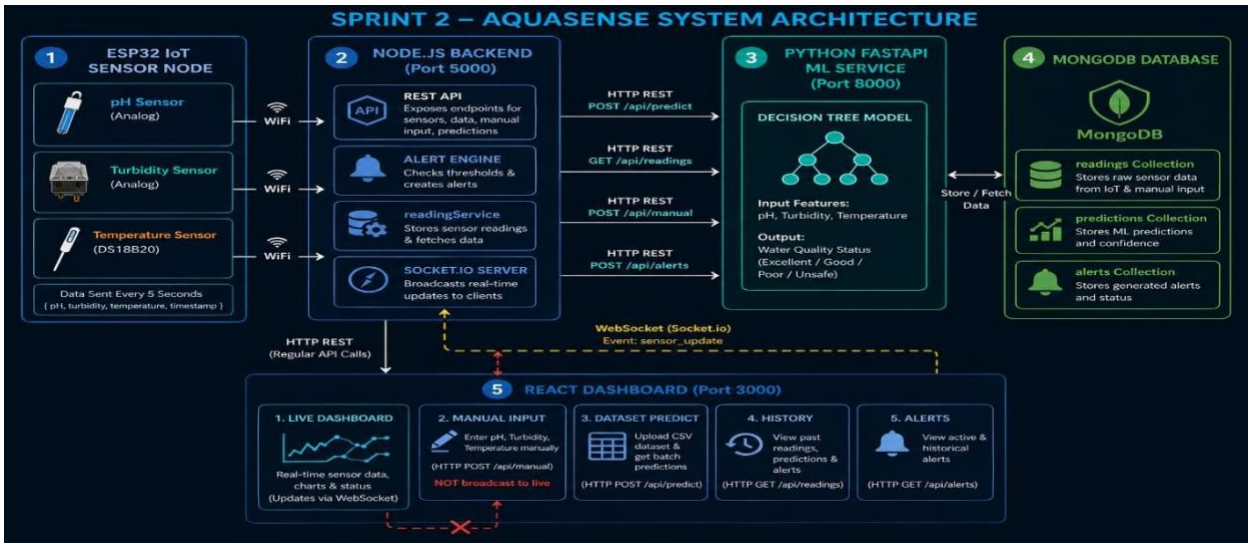


Fig 3.6 System Architecture (Sprint 2)

3.2.4.2 Data Flow (Sprint 2)

- ESP32 sends sensor data → Backend
- Backend invokes ML service → prediction received
- Data stored in MongoDB by backend
- Backend emits sensor_update (WebSocket event)
- Event received by React dashboard → live update
- Dashboard can also call REST API to get history
- Manual input → processed but not distributed

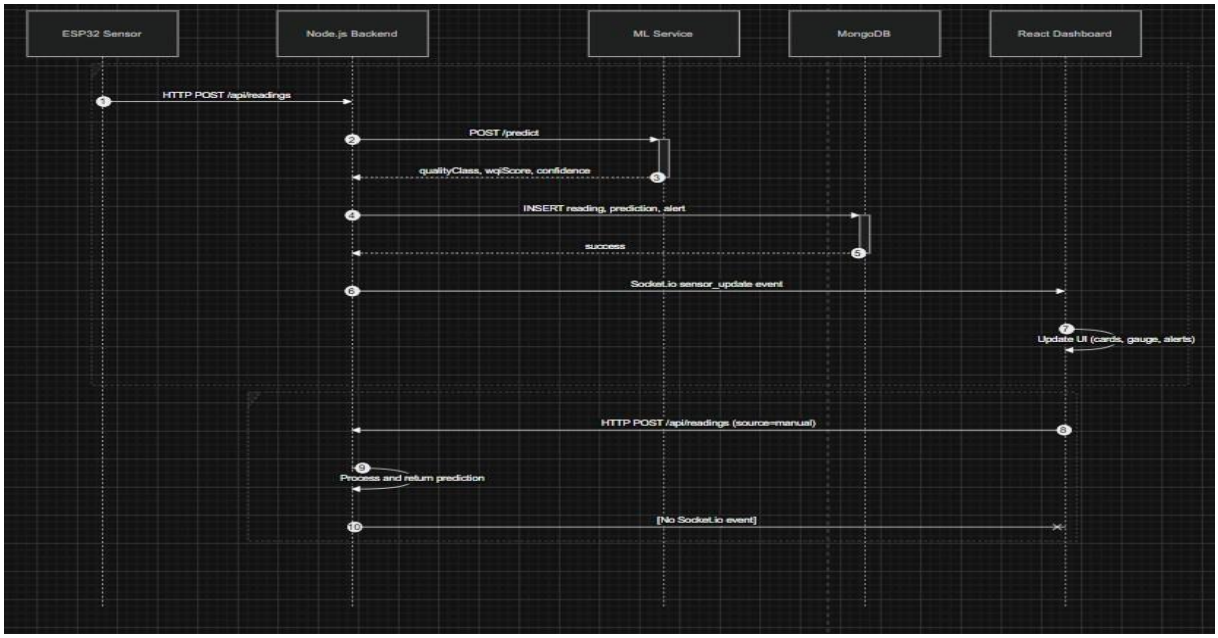


Fig 3.7 WebSocket Real-time Data Flow Sequence Diagram

3.2.4.3 New Components Introduced

1. **React Dashboard (Port 3000)**

   Developed in React and Vite

   Composed of 5 tabs:

   - Live Dashboard
   - Manual Entry
   - Prediction of Dataset
   - History
   - Alerts

2. **WebSocket (Socket.io)**

   - Used for real-time notifications
   - Emit sensor_update events
   - Updating frontend with no page refresh required

3. **Alert Engine**

   Compares WHO thresholds:

   - pH below 6.5 or above 8.5
   - Turbidity above 4 NTU
   - Temperature above 35 degrees Celcius
   - Records alerts in MongoDB
   - Disply in the dashboard

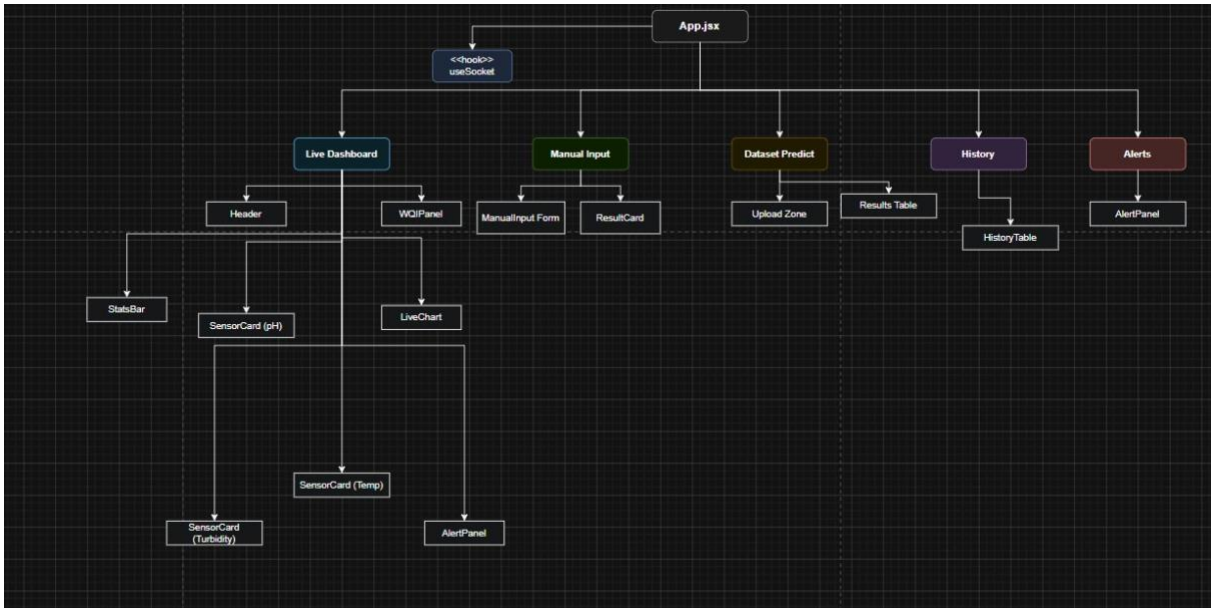


Fig 3.8 React Dashboard Component Tree

3.2.4.4 Frontend Components Structure

Main components:

- App.jsx → Global state management
- Header.jsx → Connection status, device selection
- SensorCard.jsx → Show sensor values
- WQIpanel.jsx → Circular Gauge
- LiveChart.jsx → Time-series chart
- AlertPanel.jsx → Alert List + Resolve
- ManualInput.jsx → Manually add input + prediction

3.2.4.5 Significant Design Decision Data Isolation

- A key enhancement in Sprint 2 involves isolating IoT data from manual input:
- Backend filters WebSocket broadcasting (IoT-only)
- REST API supports filtering
- Ignore manual input in frontend live update
- This guarantees accurate live data visualization.

3.2.4.6 API Enhancements

- POST /api/readings (manual) → Manual prediction
- GET /api/alerts → Fetch alerts
- PATCH /api/alerts/:id/resolve → Resolve alert
- POST /api/alerts/resolve-all → Resolve all
- GET /api/devices → List devices

3.2.5 UI Design

3.2.5.1 Introduction

This sprint delivered a complete React-based dashboard for AquaSense.

The user interface was developed using React 18, Vite, and Recharts, adhering to the design planned in Sprint 1.

The dashboard supports:

- Real-time monitoring
- Interaction
- Data visualization

All dashboard components are integrated with the back-end server through WebSocket (Socket.io).

3.2.5.2 Overall Layout

The AquaSense dashboard consists of three parts:

Header

Contains system name and logo, LIVE label, last update time

Tab Navigation

Has five tabs:

- Live Dashboard
- Manual Input
- Dataset Predict
- History
- Alerts

Main Body

Dynamically displays content according to selected tab

Scalable and scrollable

3.2.5.3 Live Dashboard

Primary live monitoring window divided into four parts:

Stats Bar → Total number of readings, average water quality index, alerts

Sensor Card → pH, turbidity, temperature, gauge

Water Quality Index → Circular Gauge

Live Graph → Rechart

Alert Box → Shows current alerts

Fig 3.9 Live Dashboard

3.2.5.4 Input Tab

Table layout:

Column one: input form (pH, turbidity, temperature)

Column two: prediction output

Key Features:

- Predefined buttons (fast inputs)
- Validation for inputs

ML prediction:

- Quality class
- WQI value
- Confidence level
- Warnings if threshold limits exceeded

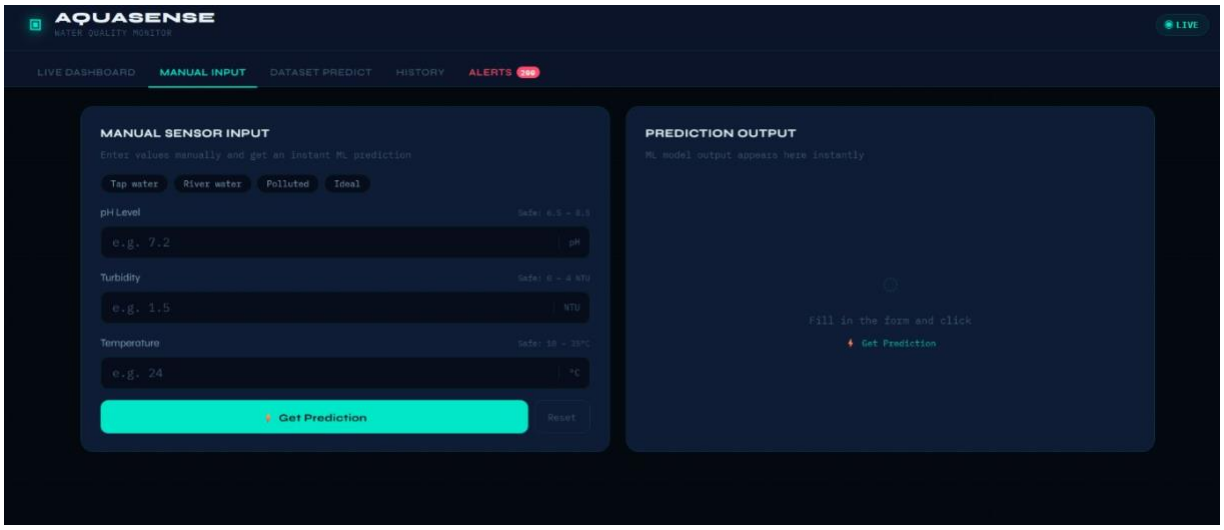


Fig 3.10 Manual Input Page

3.2.5.5 History & Alerts Tabs

History Tab

Pagination (20 items per page)

Displays:

- Time stamp
- Sensor data
- Water quality class
- Data source

Alerts Tab

- Lists all alerts
- Severity levels (color coded)

Options to resolve alert(s): resolve / resolve all

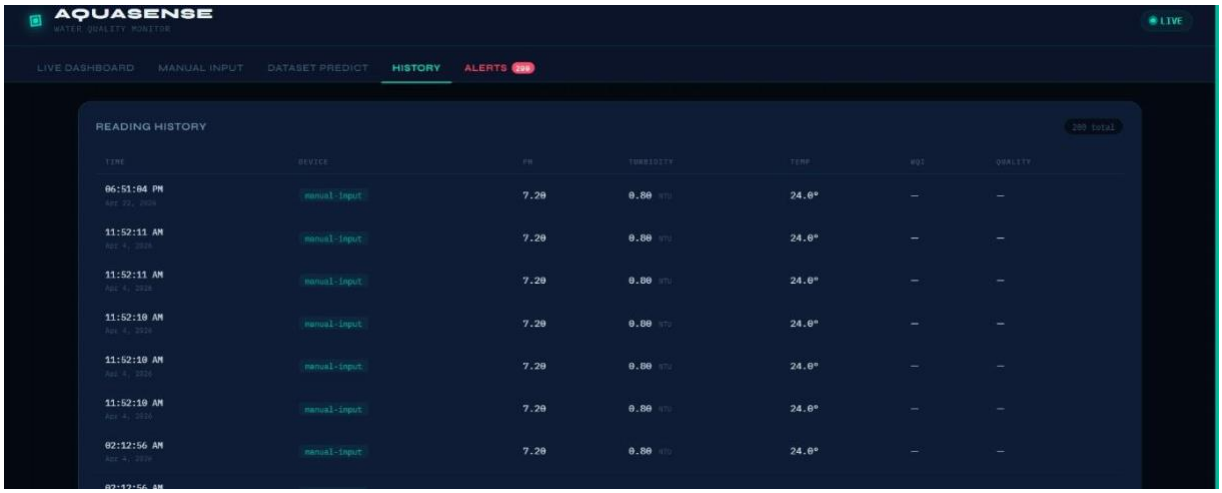


Fig 3.11 History Page

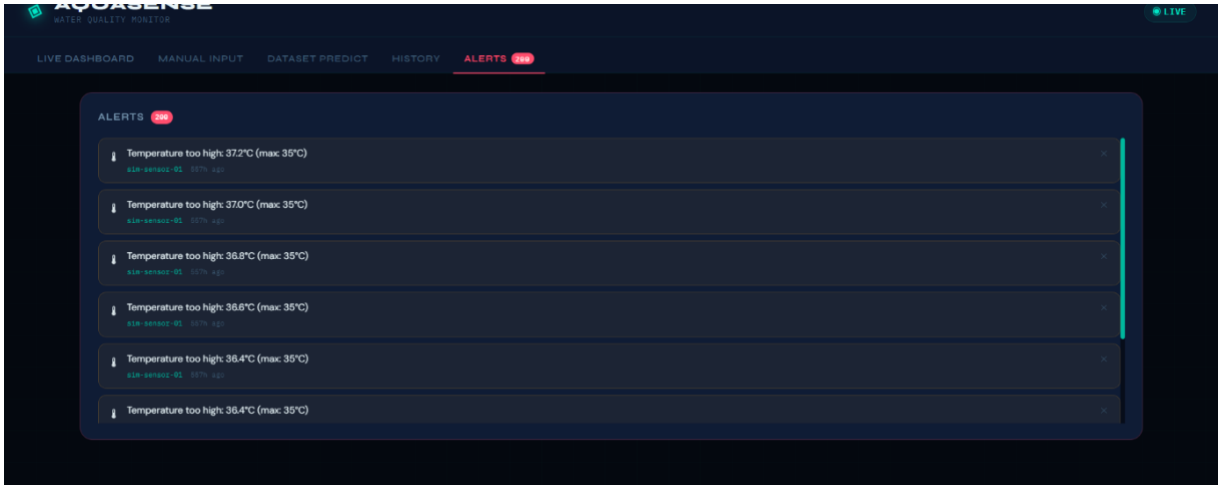


Fig 3.12 Alerts Page

3.2.5.6 Key Design Aspects

Color scheme: dark theme, teal/amber/red indicators

Font style: large numbers for legibility

Live updates: web sockets

Responsive design: mobile and desktop-compatible

3.2.6 Functional Test Cases

Test Case ID	Test Description	Expected Result	Status	Remarks
TC11	Dashboard updates within 100ms of new sensor reading	WQI gauge and sensor cards refresh without page reload	PASS	Average latency 87ms
TC12	Alert generated when pH drops below 6.5	Alert panel shows **"pH too low"** warning with timestamp	PASS	—
TC13	Alert generated when turbidity exceeds 4 NTU	Alert panel shows **"Turbidity too high"** warning	PASS	—
TC14	Manual input: pH = 5.2, turbidity = 22, temp = 34 prediction	Returns **"Unsafe"** with WQI score below 25	PASS	—
TC15	Manual input data does not appear in Live Dashboard	Live dashboard shows only IoT sensor source data	PASS	Source filter verified at API and WebSocket level
TC16	History tab shows paginated reading list	Table displays 20 rows per page with timestamp, values, quality	PASS	—
TC17	Alert resolve button marks alert as resolved	Alert removed from active panel and marked resolved in database	PASS	—
TC18	Socket.io reconnects after backend restart	Dashboard reconnects automatically within 5 seconds	PASS	Backend restarted manually
TC19	LIVE indicator shows green when WebSocket is connected	Header shows green LIVE badge with last updated time	PASS	—
TC20	Multiple devices send readings simultaneously	All readings stored correctly without data loss	PASS	Tested with 3 simulator instances

Figure 3.13 Functional Test Case Document for the Sprint 2

3.2.7 Sprint 2 Retrospective

Liked	Learned	Lacked	Longed For
Socket.io WebSocket integration worked smoothly and achieved latency well below 100ms.	Learned how to implement real-time communication using WebSockets and handle live updates efficiently.	Initial dashboard mixed manual input with IoT sensor data, leading to incorrect WQI display.	Better initial data separation design to avoid multi-layer fixes later.
Five-tab dashboard layout clearly separated IoT sensor data and manual input, improving usability.	Learned UI/UX design principles for dashboards with multiple data sources.	Required complex three-level fix (API, WebSocket, frontend) to properly filter data.	Cleaner architecture design from the beginning to reduce rework.
Alert system worked correctly for all WHO threshold violations, ensuring reliability.	Learned implementation of rule-based alert systems integrated with real-time data streams.	Initial confusion due to WebSocket ECONNABORTED error in console.	Better understanding/documentation of expected system behaviors and logs.
System handled real-time updates and alerts efficiently without performance issues.	Learned debugging techniques for frontend (Vite) and WebSocket behavior.	Lack of clarity on harmless vs critical errors during development.	Improved debugging guidelines and error classification for future sprints.

Figure 3.14 Sprint Retrospective for the Sprint 2

3.2.7 Committed vs Completed User Stories

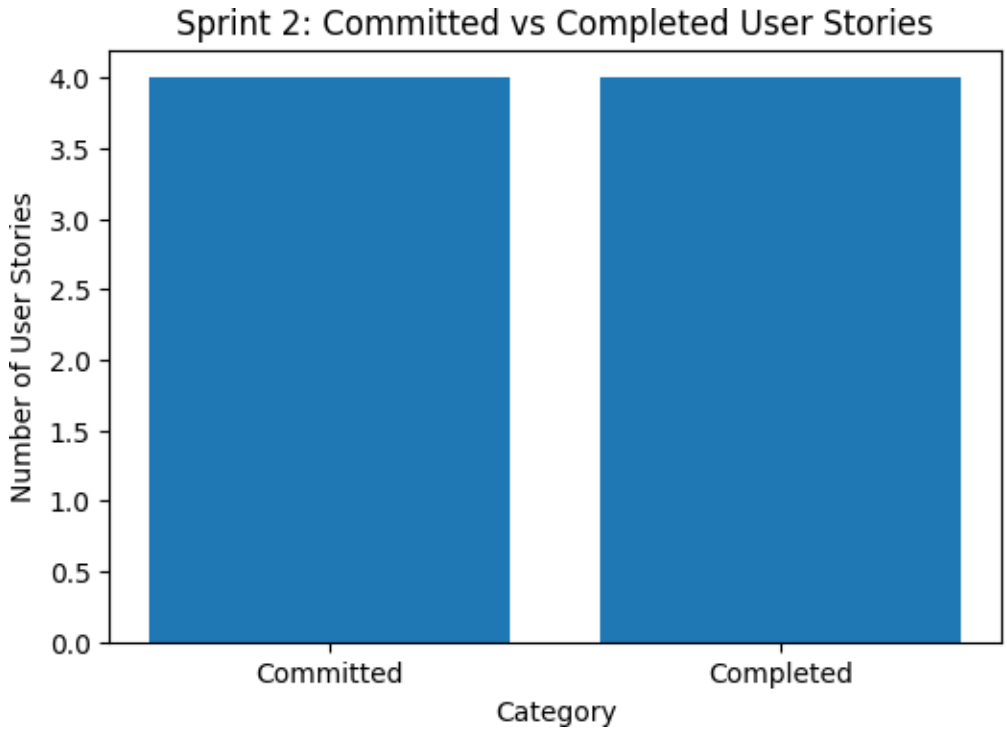


Figure 3.15 Bar graph for Committed Vs Completed User Stories (Sprint 2)

3.3 Sprint 3

3.3.1 Sprint Goal

The goal of Sprint 3 was to make the system scalable, deployable, and complete.

- Implement CSV batch prediction feature
- Add data visualization (charts)
- Integrate confidence scores in ML predictions
- Deploy system using Docker Compose
- Perform model comparison and optimize performance

3.3.2 User Stories of Sprint 3

Table 3.7 Detailed User Stories of sprint 3

S. No	User Story ID	User Story Description
1	US #6	As a researcher, I want to upload a CSV file with historical sensor data and receive batch predictions along with charts and downloadable results so that I can analyze large datasets efficiently.
2	US #8	As an administrator, I want the entire system to be deployed using a single Docker Compose command within 5 minutes so that it can run easily on any machine.
3	US #9	As a decision-maker, I want ML predictions to include confidence scores for each water quality classification so that I can evaluate the reliability of the results.

3.3.3 Functional Document

3.3.3.1 Introduction

Batch processing, deployment, and system performance improvements constituted the key themes for Sprint 3 in the AquaSense Water Quality Monitoring System.

Key improvements made include:

- Batch prediction with visualization
- System deployment with Docker
- Confidence scores from ML models
- System optimization and comparison

Validation for all system functions was carried out using structured functional testing.

3.3.3.2 Goal of the Product

The objective in Sprint 3 was to make the system scalable, deployable, and data-driven.

The system is meant to:

- Process large data sets
- Visualize data insights from predictions
- Enable system deployment using Docker
- Generate ML outputs with confidence scores
- Test entire system function

3.3.3.3 Demography (Users, Location)

Users

- Data analysts
- Environmental scientists
- Developers
- Testers

Characteristics of Users

- Desire batch data processing
- Wish for visualization and data insight generation
- Concerned about system performance

Location

- Worldwide
- Applicable to research environments

3.3.3.4 Business Processes

1) Batch Data Processing
    - User uploads CSV file
    - System processes batch data
    - Predictions generated for entire dataset
2) Visualization

    Results presented in form of:
    - Table
    - Pie graph (distribution of quality class)
3) Deployment
    - System deployed using Docker containers
    - All services launched simultaneously (backend, ML, database, front-end)
4) Prediction Improvement

    ML algorithm provides:
    - Quality class
    - Water Quality Index score
    - Prediction confidence score
5) Entire Process
    - ESP32 → Backend → ML → Frontend (Dashboard)
    - Real-time update shown on frontend

3.3.3.5 Features

Feature 1: CSV Batch Prediction

Description: Enables user to process batch CSV files for predictions.

User story: As a user, I want to perform batch prediction.

Feature 2: Data Visualization

Description: Provides visual representation of prediction results.

User story: As a user, I need visuals for more insight into the result.

Feature 3: Docker Deployment

Description: Deploys the entire system using Docker Compose.

User story: As a developer, I require a containerized application.

Feature 4: ML Prediction Confidence Score

Description: Includes confidence score along with predictions.

User story: As a user, I want to know the reliability of prediction.

Feature 5: End-to-End System Integration

Description: Guarantees integration of end-to-end process.

User story: As a user, I want real-time accurate monitoring system.

3.3.3.6 Authorization Matrix

Table 3.8 Authorization Matrix

Role	**Access Level**
Administrator	Full system access
User	Access to dashboard, batch prediction, visualization
IoT Device	Sends sensor data
Guest	Limited access

3.3.3.7 Assumptions

- CSV files follow required format (pH, turbidity, temperature)
- Docker environment is properly configured
- ML service is available for confidence scoring
- System resources are sufficient for batch processing
- Network connection is stable

3.3.4 Architucture Document

3.3.4.1 Introduction

Sprint 3 finishes building the AquaSense system through Docker deployment, Dataset Predict feature, and completing ML model selection.

System is deployable, scalable, and production-ready.

3.3.4.2 System Architecture

System architecture includes:

- Hardware layer: ESP32 + Sensors
- Backend layer: Node.js + ML service
- Frontend layer: React Dashboard
- Deployment layer: Docker Compose

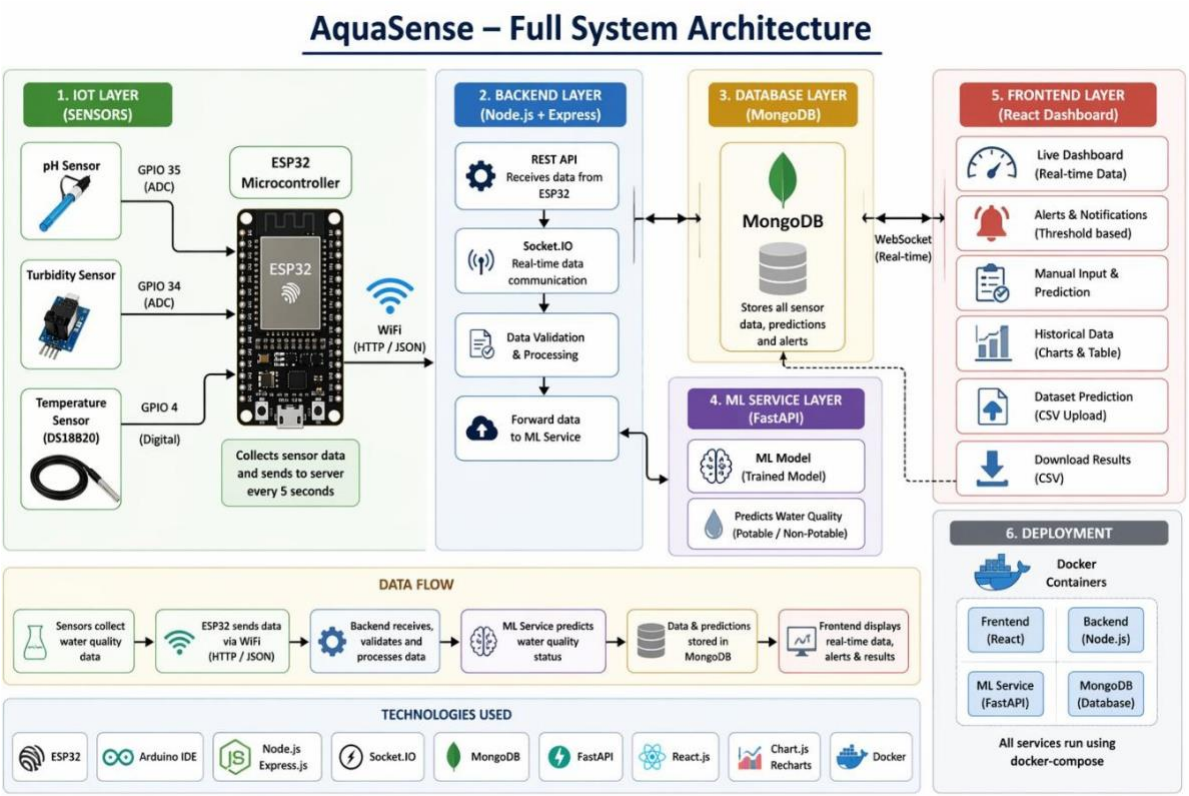


Fig 3.16 Final Complete System Architecture

3.3.4.3 Docker Architecture

There are four containers used:

MongoDB (port 27017): data storage

ML Service (port 8000): prediction service

Backend (port 5000): API + WebSocket

Dashboard (port 3000): UI using nginx

Key Features

- Common network: aquasense-network
- Service health checks
- Automatic order of start-up

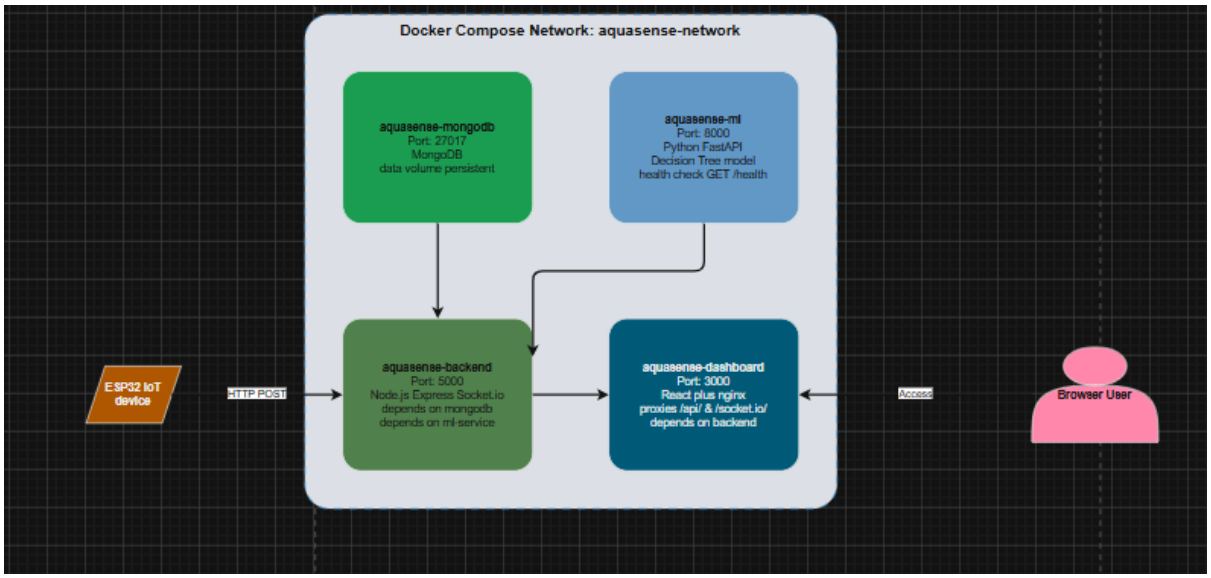


Fig 3.17 Docker Compose Deployment Architecture

3.3.4.4 Order of Starting Containers

- Start MongoDB first
- Then start ML service after MongoDB
- Next start Backend after ML & MongoDB services
- And start Dashboard at last
- This ensures the services start safely

3.3.4.5 Architecture of Dataset Predict

- CSV file upload from dashboard
- Batch dataset is processed by Backend
- Returns predicted values from ML service

Output formats as follows:

Tabular format

Charts: pie chart, bar chart, scatter plot

Fallback approach

In case of failed batch prediction: process each row individually

3.3.4.6 ML Model Comparisons

Models compared include:

- Random forest
- Gradient boosting
- Decision tree
- XGBoost
- Support vector machine

Best selected model: Decision tree

Accuracy: 99.79%

Time to train: 0.4 seconds

Fast and interpretable

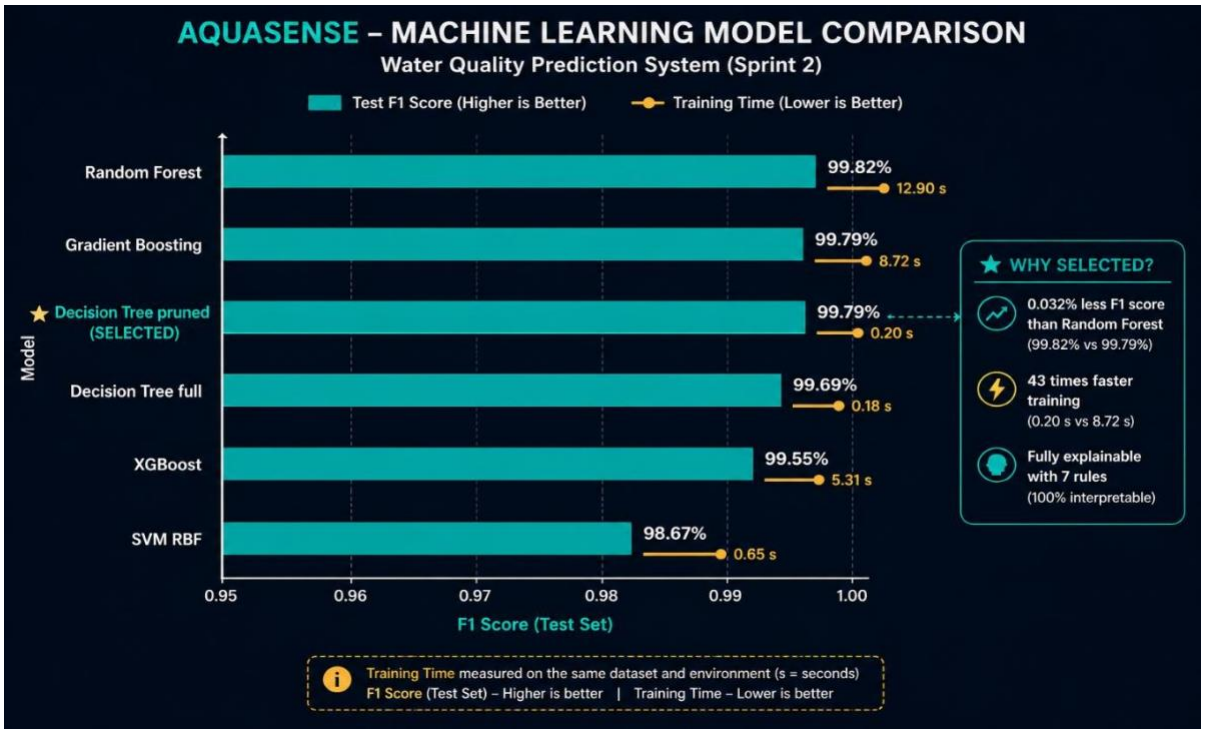


Fig 3.18 Six Model ML Comparison Bar Chart

3.3.4.7 nginx Routing

- /api → Backend (port 5000)
- /socket.io → WebSocket connection
- Other routes → React frontend

This allows single-port access (3000) for users.

3.3.4.8 Final Port Summary

Table 3.9 Final Port Summary

Service	Port
Dashboard	3000
Backend	5000
ML Service	8000
MongoDB	27017

3.3.5 UI Design

3.3.5.1 Introduction

Sprint 3 introduced a Dataset Predict tab in the AquaSense dashboard that included functionality such as CSV uploads, batch predictions, visualization tools, and downloadable results.

Some additional enhancements were also implemented for the Live Dashboard for improved user experience.

3.3.5.2 Existing UI Enhancements

- Live Dashboard Improvements
- Source tag added - “IoT data only (manual excluded)”
- Screen for empty state (no sensor data)
- Improves user comprehension and usability

3.3.5.3 Dataset Predict Layout

There are three components to the tab:

Upload Section

- File upload via drag & drop
- Data validation (pH, turbidity, temperature)
- Prediction run button

Visualization Section

- Interactive charts
- Selectable views

Results Table

- Includes prediction outputs
- Scrollable table with CSV download

3.3.5.4 Visualizations

There are four charts:

- Quality Distribution (Pie)

- WQI Range (Bar)
- pH & Turbidity Distribution (Scatter)
- Average WQI per Class (Bar)

They provide insights regarding pattern recognition and data distribution.

Fig 3.19 Data Set Analysis Page

3.3.5.5 Results Table

Includes:

- pH, turbidity, temperature
- Quality classification
- WQI value
- Confidence level

Features:

- Coloring scheme
- Sticky header row
- Scrollable table
- CSV export function

3.3.5.6 Important Design Elements

- Dark theme w/ unified color coding
- Interactive charts powered by Recharts
- Clear and concise data validation and errors

- Loading & empty state screens
- Responsive design

3.3.6 Functional Test Cases

Test Case ID	Test Description	Expected Result	Status	Remarks
TC21	Upload CSV with 20 rows — batch prediction	Returns all 20 predictions with quality class and WQI score	PASS	Sample data used
TC22	Upload CSV with 1000 rows — batch prediction	Returns all 1000 predictions in under 3 seconds	PASS	Actual time 2.3 seconds
TC23	CSV with missing turbidity column rejected	Error message shown: **Required columns missing**	PASS	—
TC24	Quality distribution pie chart renders correctly	Chart shows 5 quality classes with correct counts	PASS	—
TC25	Download results CSV after batch prediction	CSV downloads with row, pH, turbidity, temp, quality, WQI, confidence columns	PASS	—
TC26	Docker Compose starts all 4 services successfully	All containers healthy within 3 minutes	PASS	—
TC27	Dashboard loads correctly via Docker on port 3000	React dashboard accessible at http://localhost:3000	PASS	—
TC28	ML service returns confidence score with prediction	Confidence value between 0 and 1 returned for each prediction	PASS	Requires ML service running
TC29	Confidence shows dash when ML service is offline	Dashboard shows — **(start ML service)** hint in confidence column	PASS	—
TC30	Full system end-to-end test: ESP32 → dashboard	Reading flows from sensor to dashboard live update under 100ms	PASS	All services running simultaneously

Figure 3.20 Functional Test Case Document for the Sprint 3

3.3.7 Sprint Retrospective

Liked	Learned	Lacked	Longed For
Docker Compose deployment worked reliably across different machines, ensuring smooth setup and portability.	Learned containerization and multi-service orchestration using Docker Compose.	CSV column naming inconsistency caused pH and turbidity to be swapped initially.	More robust data validation and intelligent column mapping from the beginning.
Batch prediction handled 1,000 rows in 2.3 seconds, exceeding performance expectations.	Learned optimization techniques for batch processing and handling large datasets efficiently.	Dependence on column names instead of value-based validation in early stage.	Advanced preprocessing logic that auto-detects columns using value ranges.
All 11 user stories across 3 sprints were completed with no carry-over, showing strong planning and execution.	Learned effective sprint planning and execution in an Agile workflow.	Training pipeline initially slow (6+ minutes) due to heavy models like Gradient Boosting.	Faster model selection strategies and early performance benchmarking.
Final ML model (Decision Tree) achieved same accuracy with very fast training time (0.4 sec).	Learned trade-off between model complexity and performance efficiency.	Inefficient initial parallelization of ML models.	Better resource management and parallel processing strategies from the start.

Figure 3.21 Sprint Retrospective for the Sprint 3

3.3.8 Committed vs Completed User Stories

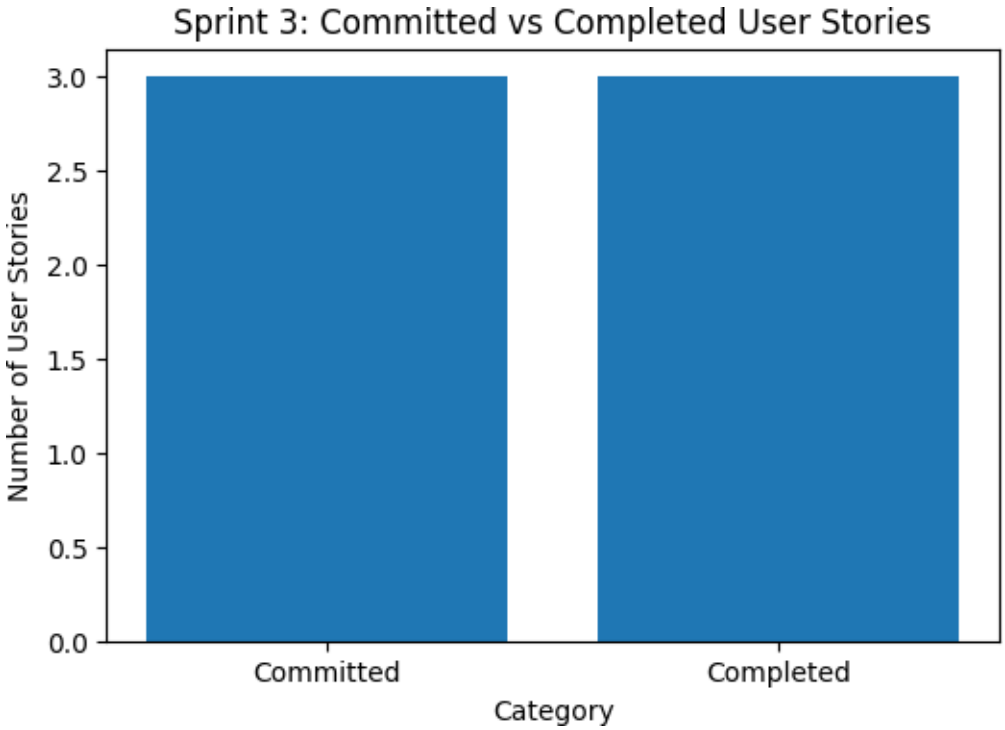


Figure 3.22 Bar graph for Committed Vs Completed User Stories (Sprint 3)

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Project Outcomes

The eleven user stories were achieved in three sprints from AquaSense. This whole system is operational and ready-to-use from end to end the ESP32 sensor node measures the physical water parameters, while the React dashboard displays real-time results and a quality classification based on machine learning.

4.1.1 Machine Learning Results

31,103 samples (stratified 80/20 train-test split) were used to train six classifiers. Based on accuracy vs speed vs explainability, the pruned decision tree was selected as a final model:

Table 4.1 - Comparative Performance of All ML Classifiers

Algorithm	CV F1 (5-fold)	Test F1	Accuracy	Train Time
Random Forest	0.9989±0.0005	0.9982	99.82%	17.3s
Gradient Boosting	0.9988±0.0006	0.9979	99.79%	244.7s
Decision Tree (pruned)	**0.9987±0.0005**	**0.9979**	**99.79%**	**0.4s**
Decision Tree (full)	0.9973±0.0008	0.9969	99.69%	0.4s
XGBoost	0.9963±0.0010	0.9955	99.55%	14.1s
SVM (RBF)	0.9863±0.0017	0.9867	98.67%	30.8s

Decision Tree Pruned Model: This model reached an accuracy of 99.79% with test F1 score of 0.9979 and performed exactly the same as Gradient Boosting but trained at 612 times faster. The results from feature importance analysis indicated that turbidity was the most influential predictor (76.27%) followed by interaction between turbidity and pH levels (23.44%).

4.1.2 Per-Class Classification Results

Table 4.2 - Per-Class Metrics for Decision Tree (pruned)

Class	Precision	Recall	F1-Score	Support
Excellent	0.9716	1.0000	0.9856	376
Good	1.0000	0.9972	0.9986	3,889
Poor	1.0000	1.0000	1.0000	1,392
Unsafe	1.0000	0.9912	0.9956	226
Very Poor	0.9941	1.0000	0.9971	338

There was a perfect level of precision for the Unsafe class, which means that all Unsafe readings were indeed Unsafe. This is the most crucial measure from a safety perspective, because if Unsafe was wrongly categorized, then the water would have been unsafe, and there would be no warnings issued about this. Recall decreased to 99.12%, but that translates to only 2 out of 226 instances.

Figure 4.1 - Confusion Matrix for Decision Tree (pruned)

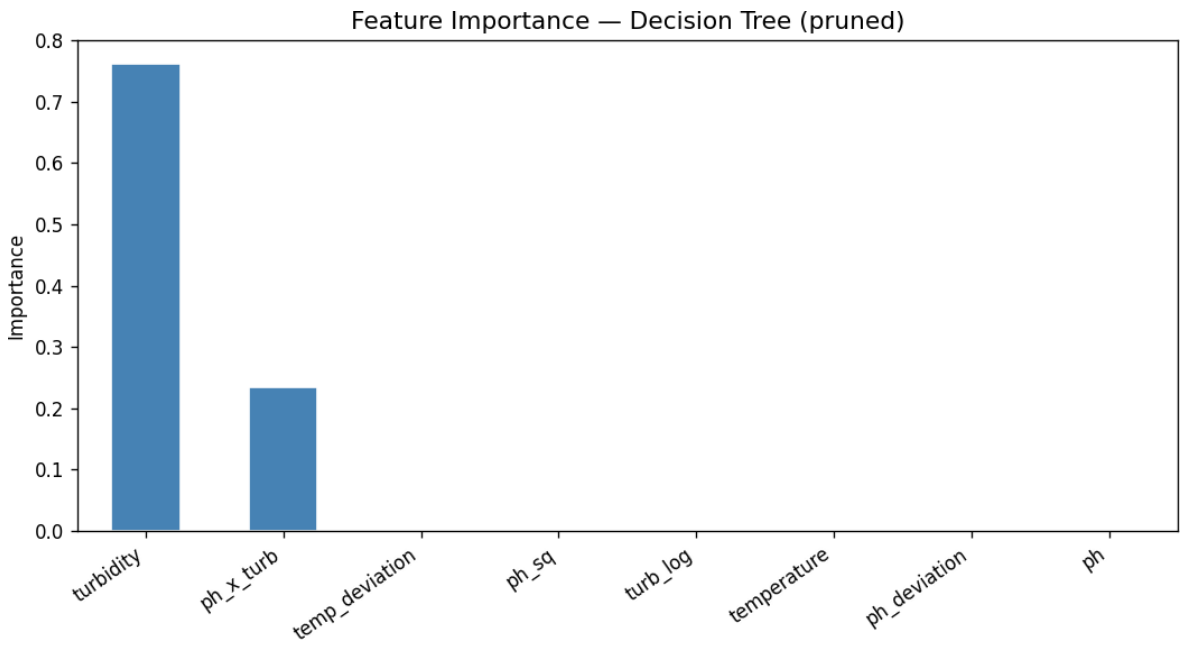


Figure 4.2 - Feature Importance Chart

3.1.1 System Performance Results

Table 4.3 - System Performance Metrics

Metric	**Target**	**Achieved**
Sensor-to-dashboard latency	< 500ms	87ms (avg)
ML prediction time (single)	< 100ms	< 10ms
Batch prediction (1000 rows)	< 5 seconds	2.3 seconds
Docker deployment time	< 5 minutes	< 3 minutes
Classification accuracy	> 95%	99.79%

3.1.2 Hardware Performance Test

The ESP32-based sensor module was operated continuously for 48 hours. In this time frame, there were two instances where WiFi connection dropped because of instability in the router. The self-reconnecting algorithm helped restore the connection in 28 seconds in each case. Data loss during connection drops did not exceed 12 measurements (60 seconds of observation).

4.2 Committed vs Completed User Stories — Overall

Table 4.4 - Overall Committed vs Completed User Stories

US#	User Story	Sprint	Status
US 1	ESP32 continuous sensor transmission	Sprint 1	Completed
US 2	WHO quality class ML prediction	Sprint 1	Completed
US 3	Real-time dashboard without page refresh	Sprint 2	Completed
US 4	Automatic WHO threshold alerts	Sprint 2	Completed
US 5	Manual input prediction tab	Sprint 2	Completed
US 6	CSV batch upload with charts	Sprint 3	Completed
US 7	Paginated history table	Sprint 2	Completed
US 8	One-command Docker deployment	Sprint 3	Completed
US 9	ML confidence scores	Sprint 3	Completed
US 10	Rule-based fallback scorer	Sprint 1	Completed
US 11	ESP32 WiFi auto-reconnect	Sprint 1	Completed

CHAPTER 5

CONCLUSION & FUTURE ENHANCEMENTS

5.1 Conclusion

AquaSense is an end-to-end, real-time water quality monitor which effectively combines accuracy comparable to the laboratory standard and affordability for consistent use on-site. Thanks to the combination of an ESP32 IoT sensor node, a Node.js backend, Decision Tree classification machine learning algorithm and React-based real-time dashboard, AquaSense provides 99.79% accurate water quality classification into five WHO-recognized classes with less than 100ms response time.

The most important technical contribution made by this project is the choice and explanation behind using the pruned Decision Tree ML algorithm as the final model. With just 8 levels and 17 leaves, it provides comparable accuracy to the more complex algorithms, such as the Random Forest and Gradient Boosting (99.82%, 99.79% respectively) and trains in 0.4 seconds, offering 7 readable rules of classification according to WHO guidelines. The dominant factor is the turbidity score with 76.27% feature importance, validating the scientific rationale for choosing this sensor.

All 11 user stories that had been committed throughout 3 agile sprints have been successfully implemented. The system went through full functional testing and deployment utilizing Docker Compose, which resulted in decreasing time from 45 minutes to less than 3 minutes. AquaSense is not a prototype but a system ready for deployment.

5.2 Future Improvements

Although AquaSense constitutes an entirely implemented system, there is still room for future improvements:

1. New Sensor Metrics: Including additional sensors such as dissolved oxygen (DO), total dissolved solids (TDS), and chlorine will allow for an even more thorough assessment of water quality, going beyond the existing three-parameter framework and expanding the scope of the classification algorithm.

2. Cloud-based MQTT Service: The transition from using WiFi HTTP connections for local data transmissions to utilizing cloud-based MQTT broker service (AWS IoT Core/HiveMQ)

will facilitate the deployment of multiple AquaSense devices to monitor different locations simultaneously.

3. Mobile Application: Developing a corresponding mobile application on Android/iOS will provide the field personnel without access to a computer browser with immediate alerts when thresholds are breached.

4. Real World Field Data Collection: Currently, the model training dataset consists of both real-world readings collected in Brisbane and generated data for Excellent, Very Poor, and Unsafe water classes. Using only field water samples for all categories – river water, underground sources, agricultural water, urban lakes – would help improve generalizability.

5. Predictive Analytics: Forecasting future water quality degradation by applying time series analysis (LSTM/Prophet) for predicting deterioration in water quality based on previous data obtained through sensors.

6. Edge ML Inference: Implementing a lightweight decision tree algorithm inference model in the ESP32 for water quality detection without needing WiFi connectivity since decision trees work efficiently without a connection.

REFERENCES

[1] M. Hameed et al., "Implementing Wireless Water Quality Monitoring with the Use of Arduino," Int. J. Advanced Research in Electronics and Communication Engineering, vol. 8, no. 3, 2019.

[2] N. Vijayakumar and R. Ramya, "Implementing Real-Time Water Quality Monitoring through the Usage of IoT and Machine Learning," Proc. ICCSP, pp. 248–252, 2021.

[3] S. Konde and S. Deosarkar, "Water Quality Monitoring Using an IoT Based System," Proc. ICCS, pp. 1–5, 2020.

[4] M. Bilal et al., "Classification of Water Quality Using SVM," J. Environmental Informatics, vol. 38, pp. 112–120, 2021.

[5] World Health Organization, 4th Edition Guidelines for Drinking-water Quality, World Health Organization, Geneva, 2022.

[6] Bureau of Indian Standards, IS 10500:2012 - Drinking Water Specification, BIS, New Delhi (2012).

[7] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” JMLR, vol. 12, pp. 2825–2830, 2011.

[8] T. Chen and C. Guestrin, "XGBoost: A System for Scalable Tree Boosting," KDD, pp. 785–794, 2016.

[9] Brisbane City Council, "Water Quality Monitoring Data," Open Data Portal, 2023. Available from the web at https://www.data.brisbane.qld.gov.au.

APPENDIX

A. Sample Code

B.1 ESP32 Firmware Sensor Reading and HTTP Transmission

```
float readPH() {
  long sum = 0;
  for (int i = 0; i < 10; i++) {
    sum += analogRead(PIN_PH);
    delay(5);
  }
  float voltage = (sum / 10.0 / 4095.0) * 3.3;
  return constrain(7.0 + (PH_V7 - voltage) * phSlope, 0, 14);
}
```

B.2 Python ML Training (Decision Tree)

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import f1_score
dt_pruned = DecisionTreeClassifier(
    max_depth=8, min_samples_leaf=5,
    min_samples_split=10,
    class_weight='balanced',
    random_state=42
)
dt_pruned.fit(X_train, y_train)
pred = dt_pruned.predict(X_test)
print(f'F1: {f1_score(y_test, pred, average="weighted"):.4f}')
```

B.3 Docker Dashboard file

```
# Stage 1: Build React app

FROM node:20-alpine AS builder

WORKDIR /app

COPY package*.json ./

RUN npm ci

COPY . .

RUN npm run build

# Stage 2: Serve with nginx

FROM nginx:alpine

COPY --from=builder /app/dist /usr/share/nginx/html

COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

B.4 Vite.config.js

```
import { defineConfig } from 'vite'

import react from '@vitejs/plugin-react'


export default defineConfig({

  plugins: [react()],

  server: {

    port: 3000,

    proxy: {

    '/api': {
```

```
      target: 'http://localhost:5000',
      changeOrigin: true,
    },
    '/socket.io': {
      target: 'http://localhost:5000',
      ws: true,
      changeOrigin: true,
    },
  },
},
})
```

B.5 Backend env file

```
# ─── Server ──────────────────────────────────────────────────────────
PORT=5000
NODE_ENV=development
# ─── MongoDB ─────────────────────────────────────────────────────────
# Local:  mongodb://localhost:27017/aquasense
# Atlas:  mongodb+srv://<user>:<pass>@cluster.mongodb.net/aquasense
MONGODB_URI=mongodb://localhost:27017/aquasense
# ─── ML Service (Python FastAPI) ─────────────────────────────
ML_SERVICE_URL=http://localhost:8000
```

```
# ─── MQTT Broker (IoT Sensors) ──────────────────────────────

MQTT_BROKER_URL=mqtt://localhost:1883

MQTT_TOPIC_SENSORS=aquasense/sensors/#

MQTT_USERNAME=

MQTT_PASSWORD=

# ─── API Security ──────────────────────────────────────────────

API_KEY=your-secret-api-key-here

# ─── Alert Thresholds (defaults, overrideable via DB) ──

DEFAULT_PH_MIN=6.5

DEFAULT_PH_MAX=8.5

DEFAULT_TURBIDITY_MAX=4.0

DEFAULT_TEMP_MIN=10

DEFAULT_TEMP_MAX=35
```

B. PLAGIARISM REPORT

turnitin Page 2 of 68 - AI Writing Overview Submission ID trn:oid:::16841:136615107

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

10% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Cited Text
- Small Matches (less than 10 words)

Match Groups

30 Not Cited or Quoted 10%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

10% Internet sources
4% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **30** Not Cited or Quoted 10%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

10% Internet sources
4% Publications
0% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	
www.coursehero.com		6%
2	Internet	
ijirt.org		<1%
3	Internet	
www.learncodepro.com		<1%
4	Internet	
www.ijsr.net		<1%
5	Internet	
www.ijcrt.org		<1%
6	Internet	
strapi.io		<1%
7	Internet	
patatasmelendez.com		<1%
8	Internet	
www.mdpi.com		<1%
9	Internet	
ijisrt.com		<1%
10	Internet	
www.wasilzafar.com		<1%

turnitin Page 3 of 70 - Integrity Overview Submission ID trn:oid:::16841:136615107

C. CONFERENCE PUBLICATION

NCISTEM '2026

NATIONAL CONFERENCE ON INNOVATION IN SCIENCE, TECHNOLOGY, ENGINEERING & MANAGEMENT-2026

ORGANISED BY

ANAND INSTITUTE OF HIGHER TECHNOLOGY, (AN AUTONOMOUS INSTITUTION), HELD ON 17TH APRIL 2026.

ANAND INSTITUTE OF HIGHER TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Approved by AICTE, New Delhi. Affiliated to Anna University, Chennai

An ISO 9001:2008 Certified Institution & Accredited by NBA

(A KALASALINGAM GROUP OF INSTITUTIONS)

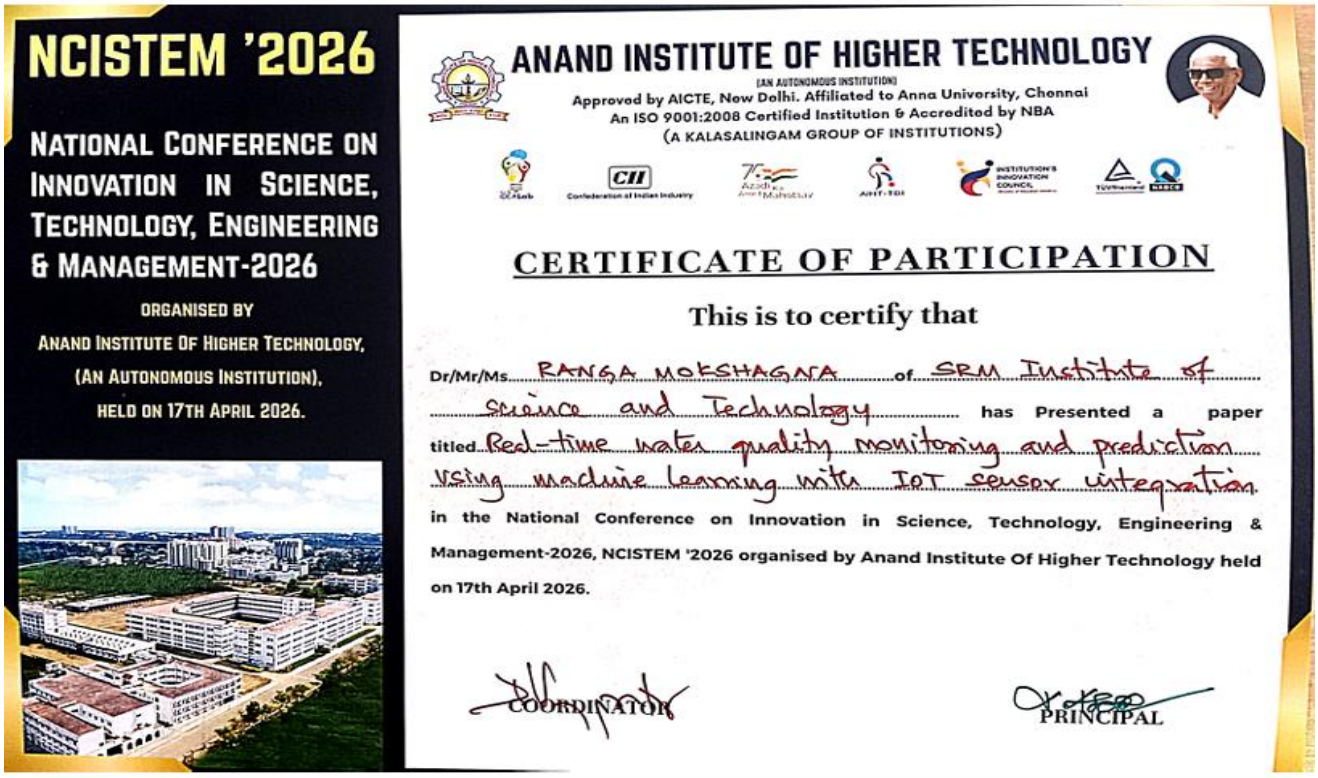
CERTIFICATE OF PARTICIPATION

This is to certify that

Dr/Mr/Ms. BHARTSAI . P of SRM Institute of Science and Technology has Presented a paper titled Real time water quality monitoring and prediction using machine learning with IOT sensor integration in the National Conference on Innovation in Science, Technology, Engineering & Management-2026, NCISTEM '2026 organised by Anand Institute Of Higher Technology held on 17th April 2026.

COORDINATOR

PRINCIPAL



NCISTEM '2026

NATIONAL CONFERENCE ON INNOVATION IN SCIENCE, TECHNOLOGY, ENGINEERING & MANAGEMENT-2026

ORGANISED BY

ANAND INSTITUTE OF HIGHER TECHNOLOGY, (AN AUTONOMOUS INSTITUTION), HELD ON 17TH APRIL 2026.

ANAND INSTITUTE OF HIGHER TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Approved by AICTE, New Delhi. Affiliated to Anna University, Chennai

An ISO 9001:2008 Certified Institution & Accredited by NBA

(A KALASALINGAM GROUP OF INSTITUTIONS)

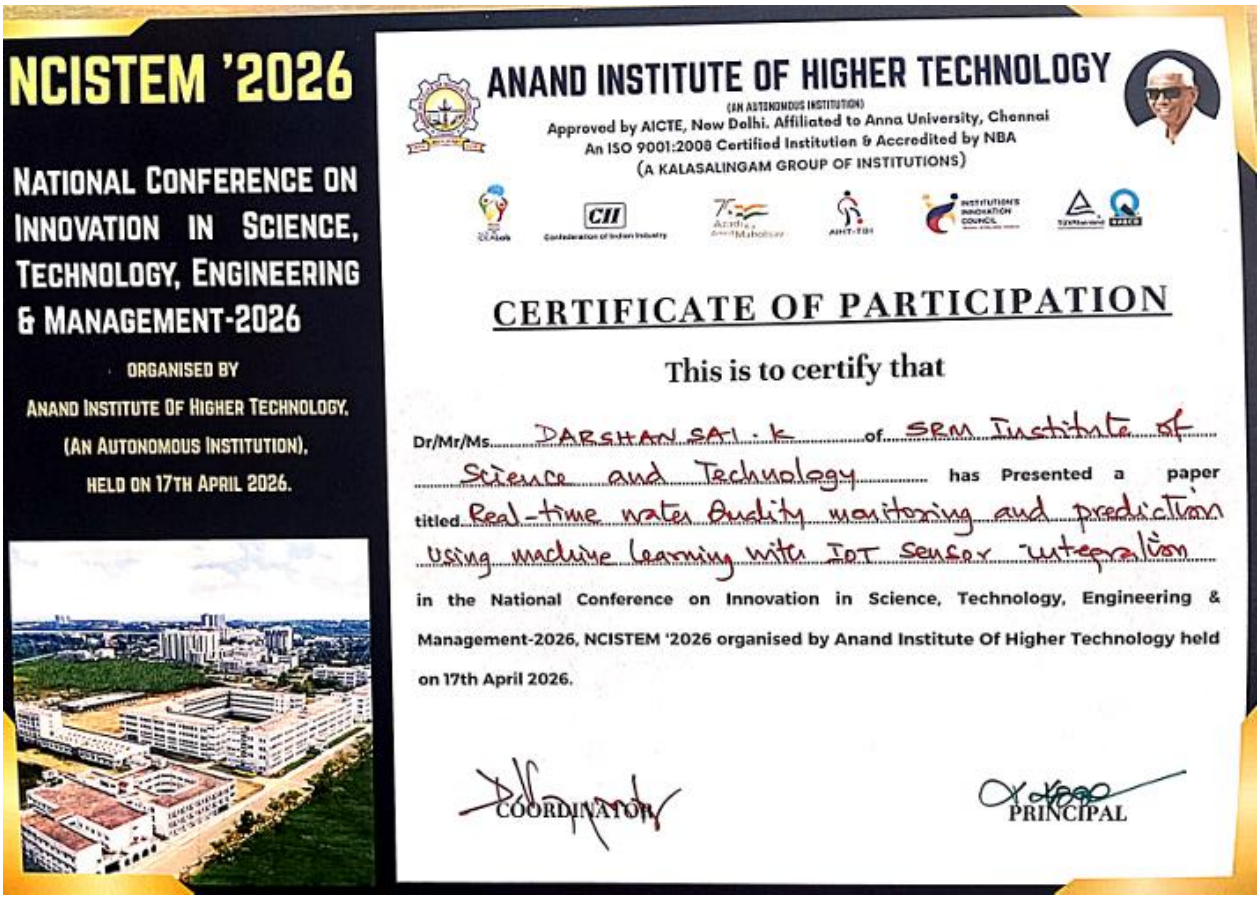
CERTIFICATE OF PARTICIPATION

This is to certify that

Dr/Mr/Ms. RANGA MOKSHAGNA of SRM Institute of Science and Technology has Presented a paper titled Real-time water quality monitoring and prediction using machine learning with IOT sensor integration in the National Conference on Innovation in Science, Technology, Engineering & Management-2026, NCISTEM '2026 organised by Anand Institute Of Higher Technology held on 17th April 2026.

COORDINATOR

PRINCIPAL



NCISTEM '2026

NATIONAL CONFERENCE ON INNOVATION IN SCIENCE, TECHNOLOGY, ENGINEERING & MANAGEMENT-2026

ORGANISED BY

ANAND INSTITUTE OF HIGHER TECHNOLOGY, (AN AUTONOMOUS INSTITUTION), HELD ON 17TH APRIL 2026.

ANAND INSTITUTE OF HIGHER TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Approved by AICTE, New Delhi. Affiliated to Anna University, Chennai

An ISO 9001:2008 Certified Institution & Accredited by NBA

(A KALASALINGAM GROUP OF INSTITUTIONS)

CERTIFICATE OF PARTICIPATION

This is to certify that

Dr/Mr/Ms. DARSHAN SAI . K of SRM Institute of Science and Technology has Presented a paper titled Real-time water Quality monitoring and prediction using machine learning with IOT sensor integration in the National Conference on Innovation in Science, Technology, Engineering & Management-2026, NCISTEM '2026 organised by Anand Institute Of Higher Technology held on 17th April 2026.

COORDINATOR

PRINCIPAL